

**REACH YOUR GOALS AND AVOID UNCERTAINTY:
CERTIFIED NEURAL LYAPUNOV-BARRIER CONTROL
FOR UNKNOWN DYNAMICAL SYSTEMS**

SUBMITTED TO THE DEPARTMENT OF COMPUTER SCIENCE OF AMHERST COLLEGE

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

FOR THE DEGREE OF BACHELOR OF ARTS WITH HONORS

MAY 5, 2026

AUTHOR: Javier Gutiérrez Bach

ADVISOR: Prof. Haoze Wu

COPYRIGHT © 2026 JAVIER GUTIÉRREZ BACH

Chapter 1

Abstract

Deep Reinforcement Learning has enabled the training of high performance controllers in robotics, game playing, and control dynamical systems, among other use cases. Despite this, the controller policies obtained remain a black-box to users, with no way to interpret their computations and no guarantees on safety and stability. Recently, there has been a line of work on training the neural controller simultaneously with a Lyapunov Barrier certificate parameterized by a neural network. The validity of the certificate can be formally verified and implies the desirable safety and liveness properties.

One challenge this approach faces is that formal verification of the certificate requires full knowledge of the true system dynamic, while in practice, the true dynamic is often not known a priori. To mitigate this challenge, this work proposes a neuro-symbolic framework that produces a controller-certificate pair for a black-box dynamics, based on a surrogate model represented by a neural network and learned from environment sampling. We integrate the surrogate dynamic into a counterexample guided inductive synthesis (CEGIS) pipeline to jointly train and verify a pair of networks that solves a Reach-While-Avoid task.

One main issue arises with our learned certificate: the proof we provide validates the agent’s deployment on the learned neural dynamic and not on the true system. We develop a theoretical framework that leverages Lipschitz continuity and sampling density to bound the error of the surrogate dynamic, and attain a robust certificate that is valid under the real dynamics. However, simple Lipschitz analysis is not enough to provide a full guarantee, as we observe that in high dimensional systems, extending the guarantee would require exorbitant amount of sampling points unfeasible to train on. Thus, the guarantees we provide are conditional on a Lipschitz constant assumption for the true system and a sensible error bound for the trained dynamic.

We evaluate our workflow on three popular control benchmarks and simulate trajectories to test whether our training improves the liveness and safety property satisfaction rate, in comparison to training on the true dynamics. Our results demonstrate that certificates are able to be synthesized in time similar to that of the true system, and also poses a great improvement in maintaining safe trajectories. The liveness goal proved to be easy to satisfy even for the initial RL controllers, and no change was seen on the controllers produced by our approach. These outcomes exhibit promise in obtaining provably safe controllers for unknown systems and highlight a future line of work on extending formal guarantees.

Chapter 2

Acknowledgements

Throughout the course of completing my thesis project and undergraduate career, I have been supported, inspired, guided, and accompanied by many wonderful people. It would be impossible to mention everyone who has contributed to my personal and professional growth over the past few years, but I am forever grateful for the impact they have had.

To begin, I would like to thank my computer science advisor, Haoze Wu, who also guided me throughout my thesis journey. I first met Prof. Wu in the spring of my junior year, when I decided to take his elective, Automated Reasoning. This was my first exposure to formal methods and verification, where my main interests in computer science and pure mathematics intersected. I found automated logic solving to be fascinating and decided that I wanted to work on something related for my thesis. Upon speaking with him, we devised this project, aiming to safeguard machine learning and robotics systems that affect real-world scenarios by guiding training with formal methods. Since then, I have improved in managing long-term research projects, communicating technical work to a variety of audiences, and understanding and adapting state-of-the-art codebases. I greatly appreciate the confidence that Prof. Wu instilled in me while embarking on what I believe was a very complex and challenging project.

I would also like to thank my previous research advisor, Shira Wein. Working with her, I embarked on my first research project aimed at a conference. Before this project, I had doubts about whether I would be capable of working in cutting-edge NLP research. However, her constant encouragement helped me recognize my ability to succeed in this field. She taught me the importance of initiative and discipline when conducting research. I also greatly enjoyed working in this lab with my co-author and peer, Emma Markle, who made the process more enjoyable and lighthearted, and who persevered through all of the obstacles with me.

I thank Lillian Pentecost, my previous academic advisor. I joined her lab the summer after my freshman year, where I conducted research for the first time. Under her guidance, I learned to use data science tools to tell a story through statistics and results. She also introduced me to reading scholarly articles and helped me overcome the initial learning curve. Although I did not continue in her research area of hardware engineering, I greatly appreciate all the academic and career advice she has offered, as well as the skills she helped me develop.

Moreover, I would like to extend my gratitude to all Amherst College faculty and professors who have shaped my education and development. I also thank Amherst College for providing me with four incredible years full of experiences and opportunities.

In a personal level, I want to thank my friends from college for letting me party and enjoy myself in times of stress, applications, and a never-ending thesis. In particular, I thank Eliza, Muskaan, Sorelle,

Sofia, Rebekah, Dylan, Morgan, and Sophie, among many others who have gifted me with great memories and laughter. I also must thank my friends from Costa Rica, who have graciously remained in contact throughout my time in the US. Mariana, Nanne, Sergio, Mar have been my long-distance friends the last couple of years, and I appreciate their long lasting support.

Most importantly, I thank my family for their unwavering support. My parents have provided me with endless and unconditional guidance in both academic and career decisions, and have supported me through every challenge I have faced. I thank my sister, brother-in-law, nephew, and grandparents for always being by my side.

Contents

1	Abstract	iii
2	Acknowledgements	v
3	Introduction	1
4	Related Work	3
4.1	Control Certificates	3
4.2	Verification	4
4.3	Dynamical Systems	5
4.4	Reach While Avoid Tasks	6
4.5	Lipschitz Continuity	6
5	Background Information	7
5.1	Notation	7
5.2	DNNs, Verification, Dynamical Systems	7
5.3	Controller Properties	9
5.4	Lyapunov Control	11
5.5	Certificates for the RWA Task	12
5.6	Mathematical Terminology	15
6	Methodology	17
6.1	Training a Surrogate Dynamic	17
6.1.1	Sampling	18
6.1.2	Training	18
6.2	Filtered Reach While Avoid Certificates	19
6.2.1	FRWA Training	19
6.2.2	FRWA Data Sampling	21
6.2.3	FRWA Verification	21
6.3	CEGIS Loop	22
6.4	Lipschitz Conditional Guarantees	23
6.5	Empirical Experiment	29

7	Case Studies	31
7.1	Two Dimensional Docking	31
7.1.1	System Dynamics	32
7.1.2	Constraints and Conditions	33
7.1.3	Experimental Setup	34
7.1.4	Results	35
7.2	Inverted Pendulum	37
7.2.1	System Dynamics and Constraints	38
7.2.2	Experimental Setup	39
7.2.3	Results	40
7.3	Two Dimensional Quadrotor	42
7.3.1	System Dynamics and Constraints	42
7.3.2	Experimental Setup	43
7.3.3	Results	45
7.4	Discussion	47
8	Conclusion & Future Work	49

Chapter 3

Introduction

Reinforcement learning algorithms have been widely used to train controllers for robotics and control systems. However, there is a common problem that has been discussed by the community: the neural networks that represent these controllers are a complete black-box, with no real interpretability on how the models reason and decide how they act. Due to this lack of transparency, there is no way us users can ensure that they behave correctly at all times, specially in scenarios that risk danger to those involved in its deployment. This issue has persisted and led researchers to develop frameworks to provide this insurance such as Lyapunov barrier functions. These functions encode properties related to the controller policy and the state space to make sure that systems remain safe after deployment and will eventually attain a desired goal. With more complex problems arising, the synthesis of useful Lyapunov functions became more difficult as the current ones were too simple. For this reason, research has focused on making these functions out of neural networks, called neural Lyapunov barrier (NLB) certificates. Proving that the certificate properties hold is non trivial for neural structures, so the use of verification engines for DNNs using methods like SMT solving has become popular in the field.

While NLB certificates enable safety assurances for Deep RL controllers, they require white box access to the system dynamics, which one does not typically have in real-world applications. Because of this, the main motivation for this work is to investigate a method to obtain certificates in these cases and the guarantees that we can provide at deployment in the real world. Assuming that we have no knowledge of the transition dynamics, and that we solely have black-box access to evaluate the next state given an

initial state and a controller action, our contributions are:

- A neuro-symbolic approach to approximate true dynamical systems and replace the transition function in controller-certificate pair synthesis frameworks;
- A theoretical analysis of the ability to leverage Lipschitz continuity and sampling density to extend certificate guarantees to deployment under true system dynamics;
- Empirical simulations of the controllers trained with our approach on three well known control benchmarks: 2D docking, inverted pendulum, 2D quadrotor.

Chapter 4

Related Work

4.1 Control Certificates

Work in certificates towards the verification of properties in control dynamical systems has grown significantly in different application areas. Dawson et al. [27] describe efforts made within robotics, particularly in synthesizing safe controllers for complex non-linear systems with its benefits and challenges. It discusses Lyapunov and Barrier functions, as well as contraction metrics which encode trajectories converging to a desired objective trajectory. These control certificate approaches have been used in other disciplines that differ from our work, such as energy management [44], biomedical systems [28], large scale networked systems [87], and chemical processes [76].

The history of Lyapunov-based certificates dates back to the 19th century [58], and significant research has been done to analyze the potential benefits and drawbacks of their computation [38]. The difficulty in synthesizing these certificates has made unscalable methods the main efforts for constructing them. These methods include manual design depending on their domain [19], [23], sum-of-square approaches that work exclusively in polynomial systems [45], [55], and finally, quadratic programming [51].

Despite the fact that this type of certificate provides important benefits, their computational intractability has prevented practitioners from obtaining new experimental results. Due to this, recent efforts have investigated more scalable methods for certificate synthesis that are independent from domain and interdisciplinary. The main certificate structure to remain scalable has been neural certificates, learned from

online or offline data. As the number of data points in the domain increases, the number of breaches of the certificate with its required constraints converges to 0 [16]. Early works exploring Neural Lyapunov Control (NLC) include [21], [46], [20], and [27]. Data-driven certificates have also been investigated in other settings such as multi-agent control [63], neural radiance fields [71], and dynamic obstacle avoidance [84]. Data driven methods cannot ensure that certificates formally satisfy all Lyapunov conditions and must thus be tied with verification to obtain rigorous guarantees.

Building on neural certificates, new methods focus on the expansion of regions of attraction (ROAs), using relaxations of restrictive constraints and adversarial training for state and output feedback loops [81]. Other ways to encourage region expansion include physics informed neural networks with rollouts [74], and Zubov sampling, which iterately samples new points near the ROA boundary to expand it and finetune the network as necessary [52], [60].

Another scalable method for certificate synthesis involves compositional certificates in complex systems. These decompose highly dimensional, large-scale, interconnected, or networked dynamical systems into smaller subsystems, compute local certificates, and use several composition rules to combine them into a global certificate for the whole system. The works that have experimented with this method are [10], [50], and [56], among others.

4.2 Verification

As discussed, many methods have been developed to construct such control certificates, but to rely on these, we need to provide formal proofs that all the Lyapunov constraints are met, so that they accurately solve the RWA task. Initial efforts for certificate verification involve Satisfiability Modulo Theory (SMT) [12], which uses first order logic to prove queries. Abate et al. [2], Ahmed et al. [4], Peruffo et al. [61] provide counterexamples when conditions are violated that are used to retrain the neural certificate, similar to the current CEGIS approach commonly used in control research [57], [37], [3]. One of the first formal tools published for verifiable certificate computation is FOSSIL [1, 29], which combines methods to verify Lyapunov and Barrier functions to obtain more comprehensive certificates that follow both properties. These methods require every constraint to be verified in the entire state space, which can be computationally

intractable.

With the rise of neural certificates, various tools that take advantage of SMT methods have been developed to verify the properties of the functions they represent. The first major neural verification engine was Reluplex [47], which works on linear constraints and ReLU activations. Its successor, Marabou [48, 78], adds better constraint propagation and case splitting, adding scalability for larger networks. These tools have been used in research for various use cases, such as the verification of binarized neural networks [9] or the verification of network abstractions [31]. Additionally, dReal is another SMT-based verification tool that works for nonlinear real arithmetic and supports continuous dynamics [36]; it was a foundational building block in NLC [21].

Other neural network verification methods that diverge from SMT include CROWN and α, β -CROWN [85], [32, 75]. These are optimization-based verifiers that prove properties by computing bounds on the outputs of the network and replacing nonlinearities with linear relaxations. The latter is an expansion that allows for tighter bounds using branching and optimization, and becomes more scalable to be competitive with SMT verifiers. The uses of the tool include [88], where networks are shrunk to expedite verification, and for verification in the ROA expansion framework [81].

4.3 Dynamical Systems

Being able to synthesize a valid controller certificate pair for a system is highly dependent on the mathematical nature of the dynamics. Wu et al. [79] extend NLC to apply to discrete-time systems. Additionally, other expansions have been made to support highly-dimensional systems [86], hybrid systems [22], and stochastic systems [5]. Moreover, the knowledge that we have on the systems behavior fully dictates our ability to produce a valid certificate. Zhou et al. [89] expand on NLC to provide Lyapunov stability in unknown nonlinear systems. In another work, Gaussian processes are used to learn the unknown part of the dynamic and use high confidence bounds to obtain a certificate valid under all possible dynamics consistent with the data [14]. Some other works that discuss using surrogate models for black-box access dynamics include [62], [53], [40], and [6].

4.4 Reach While Avoid Tasks

Many of the methods discussed above focus merely on Lyapunov stabilization or provide guarantees that do not necessarily adhere to the liveness and safety properties required by RWA tasks. However, similar control theoretic principles can be used to solve these tasks. Dawson et al. [26] learn a Lyapunov Barrier certificate to provide both stabilization and safety guarantees in a robust manner. Another important method relates to the Hamilton-Jacobi (HB) reachability, which allows for verification of optimal control and safety in dynamical systems [11]. This method has been employed in various works to solve reach-avoid tasks [34], [43], and [68]. This type of task in robotics is often related to safe reinforcement learning problems [18]. This is because maximizing the rewards obtained assimilates the liveness goal, while minimizing costs parallels the safety constraints. There have been efforts involving Lyapunov techniques to solve these problems [24], [82].

4.5 Lipschitz Continuity

In our work, we assume Lipschitz continuity of the dynamics, a standard condition in nonlinear control [49]. This property has been used in the broad area of neural network verification. For example, Weng et al. [77] provide tight bounds on local Lipschitz constants around a point with their Fast-Lip tool to efficiently certify robustness. Tsuzuku et al. [72] also use Lipschitz to bound input perturbations and ensure outputs do not change to adversary effects. Lipschitz work has also extended to the more focused area of Lyapunov control. In [15], Lyapunov conditions are verified locally, and by using Lipschitz constant bounds and intricate sampling, they are able to extend guarantees globally in low-dimensional nonlinear systems. There is further work that leverages Lipschitz continuity in the control theoretic realm, including [73], [81], [14], and [7].

Chapter 5

Background Information

5.1 Notation

We refer to the l_p norm of a vector x as $\|x\|_p$ and the spectral norm of a matrix A as $\|A\|_2$. The ball around x of radius δ with respect to the l_p norm is $\mathcal{B}_{\delta,p}(x) = \{x' \mid \|x' - x\|_p \leq \delta\}$. In all of our settings, $\mathcal{X}$ denotes the entire state space, $\mathcal{U}$ is the set of all possible control inputs, and $f : \mathcal{X} \times \mathcal{U} \rightarrow \mathcal{X}$ is a function that defines the system's transition dynamics to the next state, given an input state and action. A controller's policy $\pi : \mathcal{X} \rightarrow \mathcal{U}$ produces an action based on a given state. The action taken by the policy at state x is notated as $u = \pi(x)$. A neural model that predicts the systems transition dynamics based on empirical data is written as $\hat{f} : \mathcal{X} \times \mathcal{U} \rightarrow \mathcal{X}$. $\mathcal{X}_{\mathcal{I}}, \mathcal{X}_{\mathcal{G}}, \mathcal{X}_{\mathcal{U}}$ refer to the set of all initial, goal, and unsafe states respectively. Moreover, the set of all finite trajectories in $\mathcal{X}$ is $\mathcal{X}^*$, and the set of all infinite trajectories is $\mathcal{X}^\omega$.

5.2 DNNs, Verification, Dynamical Systems

Deep Learning. Recent advances in machine learning involve deep neural networks (DNNs) [54], structures consisting of layers of neurons that perform a linear algebra based transformation on its inputs. They aim to model complex, usually non-linear relationships with minimal error. In this work, we consider controllers trained via deep reinforcement learning (DRL) to stabilize control systems. This paradigm trains a DNN to engineer a policy mapping states (inputs) into actions (outputs) through reward driven learning.

DNN verification. Consider a trained deep neural network N . Let P denote a precondition constraining the total input space, and let Q denote a postcondition encoding a desired property for the outputs. The DNN verification problem consists on determining whether the implication

$$P(x) \Rightarrow Q(N(x))$$

holds for all admissible inputs x . Concretely, verification engines verify whether the following first-order logic formula is satisfiable:

$$P(x) \wedge \neg Q(N(x))$$

If the formula results satisfiable (SAT), then for some input in the domain, the precondition holds but the postcondition fails, meaning the property is violated. Oppositely, if the formula results unsatisfiable (UNSAT), then the specification is guaranteed to hold over the entire domain. The verification of piecewise-linear DNNs has been proven to be NP-complete [47].

Discrete Time-Step Dynamical systems. The experiments we perform rely on dynamical systems evolving in discrete time. This type of system must have trajectories described by the following state update equation:

$$x_{t+1} = f(x_t, u_t), \tag{5.1}$$

where $x_t \in \mathcal{X}$ denotes the state at time t , $u_t \in \mathcal{U}$ is the action taken by the controller at state x_t , and $f : \mathcal{X} \times \mathcal{U} \rightarrow \mathcal{X}$ is the state transition map. Said transition map, f , is generally the closed form formula for a dynamic system governed by ordinary differential equations (ODEs).

The input actions are generated by a feedback control policy $\pi : \mathcal{X} \rightarrow \mathcal{U}$, so that $u_t = \pi(x_t)$. From this logic, the closed-loop dynamics become

$$x_{t+1} = f(x_t, \pi(x_t)). \tag{5.2}$$

For our purposes, the policy π is parametrized by a DNN trained with DRL methods. This type of controllers is popular in the field of control as they are said to perform well in complex, high-dimensional

real-world environments [70].

Surrogate Dynamic Model. For a given discrete time-step dynamical system with transition function $f : \mathcal{X} \times \mathcal{U} \rightarrow \mathcal{X}$ given by $x_{t+1} = f(x_t, u_t)$, a surrogate dynamic model is a parametrized function $\hat{f} : \mathcal{X} \times \mathcal{U} \rightarrow \mathcal{X}$, typically implemented as a DNN, trained from data sampled from the environment to approximate the behavior of f . The surrogate system is thus defined by:

$$x_{t+1} = \hat{f}(x_t, u_t). \quad (5.3)$$

Surrogate dynamic models are sometimes used in place of the true dynamics in downstream tasks such as verification or controller synthesis, as the non-linearity or complexity of dynamic functions might not be supported by verification engines or hinder performance. In this work, we assume that the true dynamical system is unknown, and we only possess black-box access to the transition function. That is, given an input x_t and an action u_t , we can prompt the dynamic to obtain the exact next state x_{t+1} , but we cannot access its analytical form. This makes it impossible to calculate derivatives, obtain structural guarantees on mathematical properties like smoothness, monotonicity or convexity, perform backpropagation and gradient descent, or allow for formal verification using symbolic or bound propagation methods. This motivates the use of a surrogate dynamic model as it makes all of these tasks possible, with the expense of exactness in next-state evaluation.

5.3 Controller Properties

We are considering DRL controllers that work in discrete time-steps, for which we prove both safety and liveness properties.

Safety. The safety property entails that an unsafe state is never reached. We can define this mathematically in the following way: Let $\mathcal{X}$ be the entire state space and let $\tau \subseteq \mathcal{X}^*$ be the set of all possible finite trajectories in $\mathcal{X}$. For a given definition of safety $P \subseteq \mathcal{X}$, the system is said to be safe if and only all states

in all trajectories satisfy property P :

$$\forall \alpha \in \tau, \quad \forall x \in \alpha, \quad x \models P. \quad (5.4)$$

In our experiments, we treat a violation of the safety property P as entering a predefined unsafe region $\mathcal{X}_U \subset \mathcal{X}$. The explicit condition that we check is the following:

$$\forall \alpha \in \tau, \quad \forall x \in \alpha, \quad x \in \mathcal{X} \setminus \mathcal{X}_U. \quad (5.5)$$

An unsafe trajectory is one that is finite and contains a "bad" state $x \in \mathcal{X}_U$.

Liveness. The liveness property ensures that a trajectory will eventually reach a goal state. Let $\mathcal{X}$ be the entire state space and let $\tau^\infty \subseteq \mathcal{X}^\omega$ denote the set of all infinite trajectories. A given property P is called a *liveness* property if and only if, along every infinite trajectory, there exists a state satisfying P . Formally,

$$\forall \alpha \in \tau^\infty, \quad \exists x \in \alpha \text{ s.t. } x \models P. \quad (5.6)$$

Similarly to safety, we predefine a goal region $\mathcal{X}_G \subset \mathcal{X}$, which when entered, the liveness property is satisfied:

$$\forall \alpha \in \tau^\infty, \quad \exists x \in \alpha \text{ s.t. } x \in \mathcal{X}_G. \quad (5.7)$$

A violation of the liveness property entails an infinite trajectory that does not contain "good" state.

The key difference between both types of properties is the time horizon they constrain. The safety property regulates behavior at all times, while liveness concerns the *eventual* behavior of the system. Proving the two properties is essential to ensure autonomous controllers act as intended.

Reach-While-Avoid Task. A *reach-while-avoid* (RWA) task, is a controller synthesis problem for a dynamical system, where all trajectories *avoid* unsafe states and eventually *reach* goal states. Essentially, a controller π must be trained so that the system satisfies the safety and liveness properties. Rigorously:

Given a discrete time-step dynamical system with a state space $\mathcal{X}$, a control space $\mathcal{U}$, a set of initial

states $\mathcal{X}_I \subset \mathcal{X}$, a set of goal states $\mathcal{X}_G \subset \mathcal{X}$ and a set of unsafe states $\mathcal{X}_U \subset \mathcal{X}$, where $\mathcal{X}_I \cap \mathcal{X}_U = \emptyset$ and $\mathcal{X}_G \cap \mathcal{X}_U = \emptyset$, a controller $\pi : \mathcal{X} \rightarrow \mathcal{U}$ solves the *reach-while-avoid* task if for every trajectory $\tau = \{x_1, x_2, \dots\}$ generated under $u_t = \pi(x_t)$ with $x_1 \in \mathcal{X}_I$, the following holds:

1. **Reach:** $\exists t \in \mathbb{N}$ s.t. $x_t \in \mathcal{X}_G$.
2. **Avoid:** $\forall t \in \mathbb{N}, x_t \notin \mathcal{X}_U$.

In this thesis, we attempt to solve the *reach-while-avoid* task for an unknown, black-box access discrete dynamical system.

5.4 Lyapunov Control.

Verifying liveness and safety properties over a dynamical system given a controller policy is a non-trivial task. Learning a certificate function $V : \mathcal{X} \rightarrow \mathbb{R}$ that satisfies certain mathematical constraints has been shown to imply properties about the system under a specific policy. In this work, we combine two main types of certificate functions:

Control Lyapunov Functions. A Lyapunov function gives states in the domain space a conceptual energy level that dissipates over time, eventually reaching the zero-energy equilibrium point. The objective of this type of certificate is to guarantee *asymptotic stability*: trajectories must converge to the equilibrium [39]. The existence of a Lyapunov function implies the liveness property is satisfied in the system. The certificate function provides this guarantee by adhering to these three conditions:

1. It reaches a value of α (usually 0) at the equilibrium region.
2. The function value is strictly greater than α (usually strictly positive) everywhere else.
3. It is either monotonically decreasing [20, 21] or decreases by at least a constant ϵ at every time step [35].

Control Barrier Functions. Barrier functions are the counterpart to Lyapunov functions, they are similarly energy-level-based certificates which instead aim to guarantee the safety property over the system

[8]. The existence of this type of certificate enforces that no trajectory ever enters the unsafe region in the domain [7], [13], [80]. Proof of this behavior is afforded by creating a function that satisfies:

1. All unsafe states have an energy level bounded below by a constant β .
2. The barrier function value of all other states is strictly lesser than β .

Control Lyapunov Barrier Functions. The *Control Lyapunov Barrier Functions* (CLBF) is an energy-level-based certificate that adheres to the constraints of both certificates previously discussed to guarantee both the safety and the liveness properties in the system. The creation of these certificates was revolutionary in the field of control, as it proved useful in many real-world problems due to its ability enforce properties in dynamic systems. They have since become a staple in solving reach-while-avoid tasks, as they are a key method for providing guarantees on the conditions they require to be solved [30].

5.5 Certificates for the RWA Task

For this section, we refer to the same definition of the reach-while-avoid task described above.

RWA Certificates A certificate for the reach-while-avoid task is a function $V : \mathcal{X} \rightarrow \mathbb{R}$, if for some constants α, β, ϵ with $\alpha > \beta$ and $\epsilon > 0$, the following conditions hold:

$$\forall x \in \mathcal{X}_{\mathcal{I}}, \quad V(x) \leq \beta \tag{5.8}$$

$$\forall x \in \mathcal{X} \setminus \mathcal{X}_{\mathcal{G}}, \quad V(x) \leq \beta \implies V(x) - V(f(x, \pi(x))) \geq \epsilon \tag{5.9}$$

$$\forall x \in \mathcal{X}_{\mathcal{U}}, \quad V(x) \geq \alpha \tag{5.10}$$

In this case, the tuple $(\alpha, \beta, \epsilon)$ is called the witness for certificate V .

Prior work proves that if a certificate of this nature exists with the additional constraint that it is bounded from below, then every infinite trajectory τ that begins on a state $x \in \mathcal{X} \setminus \mathcal{X}_{\mathcal{G}}$ with $V(x) \leq \beta$ will eventually reach a state in $\mathcal{X}_{\mathcal{G}}$ and avoid the region $\mathcal{X}_{\mathcal{U}}$.

Intuitively, initial states having the constraint $V(x) \leq \beta$, provides a useful buffer zone where $\beta < V(x) < \alpha$, before possibly reaching the unsafe region where $V(x) \leq \alpha$. Additionally, Equation 5.9 ensures that for all of these states in $\mathcal{X}_{\mathcal{I}}$, the energy level throughout the trajectory keeps decreasing by at least ϵ at every time-step, ensuring it can never reach the unsafe states with higher energy levels, and because of the lower bound, descent will continue until a goal state is reached.

These types of certificates can be in the form of a neural network, using training and loss engineering to adhere to these rules and attain a valid *neural certificate* [21].

FRWA Certificates The FRWA certificate approach builds upon a neural certificate, where the DNNs that conform the Lyapunov function and the controller are kept small, in order to remain scalable for verification. Training one of these pairs for complex, non-linear systems might prove very difficult with a small network, which is why *Filtered Reach-While-Avoid* (FRWA) certificates are a viable option.

FRWA are a simplification of regular neural certificates that utilize state space masks for the unsafe and the initial regions in order to hard code the energy level of the states pertaining to those. Constants c_1, c_2 are chosen so that $c_1 \leq \alpha < \beta \leq c_2$, and during the implementation of the certificate wrapper, we enforce that $x \in \mathcal{X}_{\mathcal{G}} \implies V(x) = c_1$ and $x \in \mathcal{X}_{\mathcal{U}} \implies V(x) = c_2$, which ensures that constraint 5.10 holds. Having this condition be satisfied inherently makes training and verification much easier as the unsafe region will exert no pressure on the training loss, and less queries will need to be checked in the verification stage. The filtered approach also helps satisfying condition 5.9. This is because when hard-coding a certificate value for all states in the goal region to a constant much lower than β , then any state x for which $V(x) < \beta$ and $f(x, \pi(x)) \in \mathcal{X}_{\mathcal{G}}$ immediately satisfies the descent condition. Implementation of the FRWA certificate remains simple, as a simple wrapper can be made for the DNN which first checks the two region masks to obtain a value, and if it does not apply, then the network is used for inference.

Robust Controller for a Reach While Avoid Task The concept of a robust controller stems from the idea that a controller is able to satisfy the liveness and the safety properties of the RWA task with the condition that the next states computed by the environment function at each time step may have a bounded perturbation. Formally:

Consider a controller π for a RWA task, an initial state $x_0 \in \mathcal{X}_{\mathcal{I}}$ and a perturbed closed-loop trajectory

$\{x_t\}_{t=0}^\infty$ evolving at each time-step as such:

$$x_{t+1} = f(x_t, u_t) + \delta_t, \quad u_t = \pi(x_t), \quad (5.11)$$

where $\|\delta_t\|_p \leq \delta$ for all t . Then, the controller π is said to be *robust to state perturbations* of radius $\delta \in \mathbb{R}^+$ in the ℓ_p norm if every such trajectory satisfies:

1. **Reach:** $\exists \tau \geq 0$ s.t. $x_\tau \in \mathcal{X}_G$.
2. **Avoid:** $\forall t \in [0, \tau - 1], \mathcal{B}_{\delta,p}(f(x_t, u_t)) \cap \mathcal{X}_U = \emptyset$.

Intuitively, this encodes that for any given state, the set of all possible next states in the perturbation radius remains within the safe region. The problem also proves difficult as the reachable set expands quickly at each time-step since each state in the set has its own ball of possible next states, leading to a phenomenon called *state expansion* [73].

Having a different definition for a controller that solves the perturbed RWA task, we must extend the requirements of the Lyapunov Barrier certificate to account for noisy environment functions.

Robust Lyapunov Barrier Certificate for RWA Tasks A (δ, p) -Robust Lyapunov Barrier Certificate with witness $(\alpha, \beta, \epsilon)$ for the reach-while-avoid task is a function $V : \mathcal{X} \rightarrow \mathbb{R}_{\geq c}$ with initial set $\mathcal{X}_I$, goal set $\mathcal{X}_G$ and unsafe set $\mathcal{X}_U$ if $\alpha > \beta > c, \epsilon > 0$, and the following conditions hold:

$$\forall x \in \mathcal{X}_I, \quad V(x) \leq \beta \quad (5.12)$$

$$\forall x \in \{x \in \mathcal{X} \setminus \mathcal{X}_G \mid V(x) \leq \beta\}, \forall y \in \mathcal{B}_{\delta,p}(f(x, \pi(x))), \quad V(x) - V(y) \geq \epsilon \quad (5.13)$$

$$\forall x \in \mathcal{X}_U, \quad V(x) \geq \alpha \quad (5.14)$$

The difference with this certificate comes in constraint 5.13, as it now requires the descent condition to hold for all possible next states in the perturbation ball. With parallel intuition to that of the original RWA certificate, the existence of a robust certificate logically implies that the controller will drive the system towards the goal region in finite time while maintaining safety despite the perturbations in the

environment, thus solving the robust RWA problem with formal guarantees [73].

5.6 Mathematical Terminology

Lipschitz Continuity. A function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is globally Lipschitz continuous on $\mathcal{X} \subseteq \mathbb{R}^n$ with respect to the norm l_p if there exists a nonnegative constant $L \geq 0$ such that

$$\|f(x) - f(y)\|_p \leq L\|x - y\|_p \quad \text{for all } x, y \in \mathcal{X}.$$

The smallest such constant L is called the *Lipschitz constant* of f on $\mathcal{X}$, defined as

$$L_f := \sup_{\substack{x, y \in \mathcal{X} \\ x \neq y}} \frac{\|f(x) - f(y)\|_p}{\|x - y\|_p}.$$

Following the framework developed in [73], if we have a valid, L_∞ -Lipschitz continuous, Lyapunov Barrier Certificate, V , with parameters $(\alpha, \beta, \epsilon_r)$, then the same certificate serves as a (δ, ∞) -Robust Lyapunov Barrier Certificate with parameters $(\alpha, \beta, \epsilon_r - L_\infty \cdot \delta)$. As the descent parameter ϵ must be strictly positive, then that creates a constraint based on our perturbation bound and lipschitz constant:

$$\epsilon = \epsilon_r - L_\infty \cdot \delta > 0 \implies \epsilon_r > L_\infty \cdot \delta. \quad (5.15)$$

For a feedforward neural network with ℓ layers which transforms input x into output $x_{\ell+1}$ defined by:

$$x_1 = x, \quad x_{k+1} = \phi(W_k x_k + b_k) \text{ for } k = 1, \dots, \ell - 1, \quad x_{\ell+1} = W_\ell x_\ell + b_\ell$$

The matrices $W_k \in \mathbb{R}^{n_{k+1} \times n_k}$ and the vectors $b_k \in \mathbb{R}^{n_{k+1}}$ are the learned parameters of the network, and ϕ is the activation function.

When ϕ has a Lipschitz constant of 1, then the overall smoothness of the network to input perturbations can be bounded using only the weight matrices. In particular, the global Lipschitz constant of the network with respect to the ℓ_2 norm can be bounded above by the product of the spectral norm of the weight

matrices of each level:

$$L_2 \leq \prod_{k=1}^{\ell} \|W_k\|_2, \quad (5.16)$$

where $\|W_k\|_2$ denotes the spectral norm of the k -th weight matrix. Intuitively, the Lipschitz constant of composed functions is the product of the constants of each component, and each layer is itself bounded by its spectral norm, so the global bound follows.

This method is very convenient and easy to compute, with the trade-off that the bound is conservative.

In our work, we focus on simple networks with ReLU activations with Lipschitz constants of 1.

Norm Conversion. For a given ℓ_p norm, we can upper bound the L_p Lipschitz constant of a function by scaling the ℓ_2 -Lipschitz constant by a factor depending number of input and output dimensions of the function:

$$L_p \leq K_{p,2} \cdot L_2, \quad \text{where } K_{p,2} = n^{\frac{1}{2} - \frac{1}{p}} m^{\frac{1}{p} - \frac{1}{2}} \quad (5.17)$$

The variables n and m are the input and output dimensions of the function respectively.

R-Cover. Let $\mathcal{X} \subset \mathbb{R}^n$ and let $\{x_i\}_{i=1}^N \subset \mathcal{X}$. The set $\{x_i\}_{i=1}^N$ is an r -cover of $\mathcal{X}$ if

$$\mathcal{X} \subseteq \bigcup_{i=1}^N \mathcal{B}_{r,p}(x_i).$$

Similarly, if for every $x \in \mathcal{X}$,

$$\min_{1 \leq i \leq N} \|x - x_i\|_p \leq r,$$

then $\{x_i\}_{i=1}^N$ is an r -cover of $\mathcal{X}$.

This concept will be used together with Lipschitz continuity to bound the error of our surrogate model.

It serves as a way to encode the sampling density needed when training the dynamic to bound its error.

Chapter 6

Methodology

In this section, we adapt the approach in Mandal et al. [56] that synthesizes a neural Lyapunov Barrier certificate for a Reach While Avoid task so that it works for a learned surrogate dynamic instead of the ground-truth transition function. We additionally aim to verify the safety and liveness properties under this surrogate model and extend the guarantees to the true real-world environment.

6.1 Training a Surrogate Dynamic

Obtaining a formal certificate for a dynamical system for which we only have black-box access proves to be a non-trivial process. During training, this need not be a problem, as we can query the black-box to obtain next states for all sampled state-action pairs, along with their corresponding certificate values needed to encode the Lyapunov conditions in the loss objectives. However, verification is fundamentally more demanding. Current methods that provide formal proofs rely on symbolically analyzing the relationship between the certificate of one state and the next state, which in turn requires a concrete representation of the dynamical system. Consequently, the lack of an explicit dynamical function is a restrictive barrier to our ability to synthesize a valid controller-certificate pair.

To address this challenge, we propose a neuro-symbolic approach where we replace the true dynamic function in the training and verification pipeline with a surrogate neural network that accurately predicts model behavior under the physical system. The network is trained from samples taken from the environ-

ment, exploiting the black box access that we have to the system.

6.1.1 Sampling

The dataset used to train the surrogate dynamic must fully represent the domain of interest. For this, we generate N points uniformly at random that cover the entire domain and extend lightly beyond it along each state dimension to allow the surrogate dynamic to generalize comprehensively.

The inputs to this data set are state and action pairs, and the output is the state given by the true dynamic based on these inputs. Data points generated in this manner will accurately depict how the controller’s policies on a given state will affect the system based on the true dynamical system.

It is important to note that sampling random controller actions throughout the domain space might not be fully representative of the cases that we would see in true simulation, as some of these sampled actions make no sense for a given state. For example, if we randomly sample a state that is near the boundary of the region, then a working controller would never make policies that would push the system out of the domain; however, our sampling method could generate random actions that represent this inadequate behavior. One way to avoid this problem would be to use a controller to get more representative actions in our dataset, but as our workflow aims to produce said controller, this is circular. Alternatively, the initial controller that is to be finetuned could be used, but this one is faulty and imperfect when satisfying the safety and liveness properties, so the actions sampled from this controller might not actually be representative.

6.1.2 Training

After the data is sampled, we proceed with the training of the surrogate neural network. This is done as a first step completely independent of the CEGIS pipeline used to synthesize the certificate.

Neural networks with few linear layers and ReLU activations are known to be powerful function approximators, capable of representing a range of linear and non-linear functions [42], [83]. In this work, we employ relatively shallow architectures to balance accuracy and verification complexity, as each ReLU activation creates an additional branch in the verification process, increasing the time to verify exponentially.

We train the neural network on the dataset consisting of state-action pairs and their corresponding next states obtained through queries on the black-box system, as described on the previous subsection. The network parameters are updated with the objective of minimizing the prediction error between the true and predicted next states using standard gradient-based optimization.

In addition to minimizing average prediction error, we experiment with a tail-focused loss term to penalize high-error samples more strongly. This is done as the maximum error plays an important role in providing conditional guarantees via Lipschitz, as described in Chapter 6.4. To design this objective term, we implement average top-k loss [33]. This involves a process where during each epoch we record the top k % of points with the highest mean squared error, and further optimize the parameters of our network to reduce these errors. We calculate the tail loss with the following equation:

$$L_T = M + \alpha * M^2, \quad (6.1)$$

where M is the tail MSE, α is a weighting parameter, and L_T is the tail loss.

Additionally, we weight this tail loss by a weight β and vary the number of times per epoch that we optimize parameters based on the top-k percent samples.

6.2 Filtered Reach While Avoid Certificates

6.2.1 FRWA Training

We use the same reinforcement learning objective functions as the original pipeline to train the controller and the certificate to follow the certificate constraints [56]. The main difference lies in that instead of using the real dynamic f , we calculate our loss from the predicted dynamic $\hat{f}$.

Consider the training set $\{x_i\}_{i=1}^N \subset \mathcal{X}$ with $x'_i = \hat{f}(x_i, \pi(x_i))$. To satisfy constraint 5.8, we define:

$$O_s = c_s \sum_{i|x_i \in \mathcal{X}_I} \frac{\text{ReLU}(\delta_1 + V(x_i) - \beta)}{\sum_{i|x_i \in \mathcal{X}_I} 1} \quad (6.2)$$

This loss objective penalizes the amount by which samples in the initial region go beyond β in their

certificate value. At 0 loss, the Lyapunov constraint will be met for all sample points. The parameter δ_1 serves as a margin to over-approximate the constraint, and the parameter c_s is a weight to tune the importance of each constraint in the overall loss.

For condition 5.9, we use the following formula:

$$O_d = c_d \sum_{i|x_i \in \mathcal{X} \setminus (\mathcal{X}_U \cup \mathcal{X}_G), V(x_i) \leq \beta} \frac{\text{ReLU}(\delta_2 + \epsilon + V(x'_i) - V(x_i))}{\sum_{i|x_i \in \mathcal{X} \setminus (\mathcal{X}_U \cup \mathcal{X}_G), V(x_i) \leq \beta} 1} \quad (6.3)$$

This loss function targets all points in the safe and non-goal region where the certificate value of the next state decreases by less than ϵ , or even increases. Similarly as before, constants δ_2 and c_d are used to over-adhere to the constraint and to weight the term's importance in training. Both of these training objectives are defined by calculating the average of the constraint violation over all relevant points.

Furthermore, in order to extend certificate guarantees to the true dynamic, Equation 5.15 indicates that a certificate with low Lipschitz constant benefits its robustness on environment perturbations. Using the spectral norm bound from 5.16 converted to the ℓ_∞ norm, we experiment in some case studies with the following training objective:

$$O_\ell = c_\ell \text{ReLU} \left(\delta_3 + K_{\infty,2} \cdot \prod_{k=1}^{\ell} \|W_k\|_2 - \tau \right). \quad (6.4)$$

The formula penalizes when the bound for the Lipschitz constant of the certificate is greater than τ . This objective bound can be varied as needed. The constants δ_3 and c_ℓ serve the same function as in the other training objectives. It is important to remark that even though this spectral-norm-based bound is conservative, it is much more scalable than other methods that provide tighter bounds, so it is a better fit to our experiments as we want to calculate this bound for every epoch. For experiments in which we exclude the lipschitz constraint, we set $O_\ell = 0$. The total loss per batch is the sum of the weighted loss terms described above:

$$O = O_s + O_d + O_\ell \quad (6.5)$$

6.2.2 FRWA Data Sampling

Based on the training objectives described above, we only need to sample data from the set $(\mathcal{X} \setminus (\mathcal{X}_U \cup \mathcal{X}_G)) \cup \mathcal{X}_I$. For training the controller-certificate pair, we generate both a training set and a validation set, with the latter being 10% the size as the former. Additionally, for each set, we sample 80% of $\mathcal{X} \setminus (\mathcal{X}_U \cup \mathcal{X}_G)$ and 20% of the samples only from $\mathcal{X}_I$. This is mainly because constraint 5.8 on the initial set is easier to satisfy than constraint 5.9 throughout the safe region. Moreover, points from the initial region will be included in the first set as the regions overlap, and we simply isolate the initial set to have more focused training for each constraint.

Once counterexamples have been found by the verifier, we create a new dataset, including data points sampled across the entire domain, points in the specific regions where the verifier found said counterexamples, and the counterexamples themselves along with new points sampled from a small neighborhood. This type of sampling allows the controller and certificate finetuning to fix behavior in problematic regions.

6.2.3 FRWA Verification

After each training step, we use a DNN verification engine to formally verify that conditions 5.8-5.10 for our certificate hold throughout the entire domain space. As expressed in the background chapter, the implementation of the Filtered RWA certificate allows us to hard-code the energy level values for states in the goal and unsafe regions, allowing us to skip verification of condition 5.10 as it is satisfied by construction.

Condition 5.8 is checked for the entire initial region for the learned neural network, and given the construction of our filtering wrapper, it is evident that the property will also hold for the full certificate. To prove Condition 5.9, we check the following formula:

$$\begin{aligned} \forall x \in \mathcal{X} \setminus (\mathcal{X}_U \cup \mathcal{X}_G), x' \in \mathcal{X}, \\ (x' = \hat{f}(x, \pi(x)) \wedge V(x) \leq \beta) \implies ((V(x) - V(x') \geq \epsilon \vee (x' \in \mathcal{X}_G)) \wedge (x' \notin \mathcal{X}_U)). \end{aligned} \quad (6.6)$$

By fixing the certificate value in $\mathcal{X}_U$, we ensure that the descent condition fails if the next state is in the unsafe region, so we can simply check that $x' \notin \mathcal{X}_U$. We can also soundly exclude this region from the

check as $V(x) > \beta$ in the unsafe region, so the descent condition does not apply. Similarly, if we transition into a state in the goal region, the hard-coded certificate value forces the descent condition to be trivially true.

Additionally, notice how we verify the certificate conditions on the surrogate dynamic $\hat{f}$ rather than the true dynamic f . We require this approach as verification methods are impossible with only black-box access of the real transition function.

Once a valid certificate is obtained, we check the following condition:

$$\begin{aligned} & \forall x \in \mathcal{X} \setminus (\mathcal{X}_U \cup \mathcal{X}_G), x' \in \mathcal{X}, \\ & (\|x' - \hat{f}(x, \pi(x))\|_\infty < \delta \wedge V(x) \leq \beta) \implies ((V(x) - V(x') \geq \epsilon \vee (x' \in \mathcal{X}_G)) \wedge (x' \notin \mathcal{X}_U)). \end{aligned} \quad (6.7)$$

where δ refers to the radius of perturbation of the transition function $\hat{f}$. This new formula verifies that the descent condition holds for all states $x \in \mathcal{B}_{\delta,p}(f(x, \pi(x)))$. We attempt this check, increasing δ until a counterexample is found, to find the largest perturbation radius for which the certificate is valid.

6.3 CEGIS Loop

For the training and verification of the controller and certificate we follow a Counter Example Guided Inductive Synthesis (CEGIS) approach as developed in [69], [66]. After obtaining a sufficiently accurate surrogate dynamic, we jointly train the controller and the certificate in a supervised manner based on the objective functions described in 6.2.1, with the purpose of satisfying the certificate constraints. Once we reach a loss of 0 across the samples, we use the neural network verification tool Marabou [48, 78] to verify if the constraints are satisfied throughout the entire domain. When a counterexample is found, we resample in a small neighborhood around the point to augment the training set and repeat training to make sure that the constraints are met in problematic regions of the state space. We iterate this process until we obtain a controller certificate pair that is fully verified on the domain. Figure 6.1 illustrates the CEGIS workflow as described above.

The main difference of our implementation to learn a FRWA certificate with a CEGIS loop, is that both the training and the verification process use the neural network as the dynamic rather than the ground

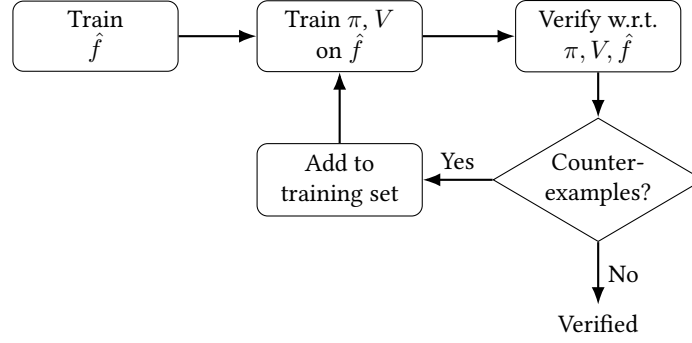


Figure 6.1: CEGIS-based training and verification workflow.

truth discrete time step formula for the system dynamics. During training, the descent term on the loss is based on violations of the epsilon from the next step generated by the controller and the learned dynamic. Similarly in the encoding of verification, instead of using the true system function to put constraints on the next state values, we include the surrogate dynamic as another network in Marabou, and then equate its outputs with the next state values. Moreover, instead of forcing equality, we can allow a small buffer in the difference between the next state and the dynamic output to account for errors of our dynamic. This allows us to verify the properties on the dynamic given a perturbation radius.

6.4 Lipschitz Conditional Guarantees

Although we are able to provide a valid certificate for a controller in our workflow, there is one caveat with our results: the certificate is valid under the learned surrogate neural network with which we replace the true dynamic system. Ultimately, we want to be able to extend these Lyapunov guarantees to the true dynamic, and answer the question of whether the liveness and safety properties will always be satisfied in real world conditions. In this section, we develop conditions for a certificate that is valid with respect to the learned dynamic in order to transfer guarantees to the true dynamic. The condition we develop assumes knowledge of the Lipschitz constants of the certificate and both dynamics. In the absence of this assumption, dynamic functions may behave arbitrarily between sampled points, and no finite set of observations would be able to bound this behavior. With no such bounds, the proof we obtain need not generalize to the true system. In this work, we are able to bound the Lipschitz constants of the certificate

and the surrogate dynamic using 5.16 since we have their white-box access, but the Lipschitz of the true system remains an assumption.

Theorem 6.4.1 (Error bound from Lipschitz continuity). *Let $\mathcal{X} \subset \mathbb{R}^n$ be a domain. Let $f, g : \mathcal{X} \rightarrow \mathbb{R}^m$.*

Assume:

1. *f is Lipschitz continuous on $\mathcal{X}$ with Lipschitz constant L_f ,*
2. *g is Lipschitz continuous on $\mathcal{X}$ with Lipschitz constant L_g ,*
3. *There exists a finite sampled set $\{x_i\}_{i=1}^N \subset \mathcal{X}$ such that*

$$\|f(x_i) - g(x_i)\| \leq M \quad \text{for all } i.$$

For any $x \in \mathcal{X}$, let x_i denote the nearest sampled point to x . If

$$\|x - x_i\| \leq \frac{\delta - M}{L_f + L_g}, \quad \text{with } \delta \geq M$$

then

$$\|f(x) - g(x)\| \leq \delta.$$

Proof. Fix any $x \in \mathcal{X}$, and let x_i be the nearest sampled point to x in the set $\{x_i\}_{i=1}^N$. Then:

$$\begin{aligned} \|f(x) - g(x)\| &= \|f(x) - f(x_i) + f(x_i) - g(x_i) + g(x_i) - g(x)\| \\ &\leq \|f(x) - f(x_i)\| + \|f(x_i) - g(x_i)\| + \|g(x_i) - g(x)\|, \end{aligned}$$

where the last step follows from the triangle inequality.

By assumptions 1 and 2 on Lipschitz continuity, we have

$$\|f(x) - f(x_i)\| \leq L_f \|x - x_i\|, \quad \|g(x_i) - g(x)\| \leq L_g \|x - x_i\|.$$

Combining these bounds with assumption 3 on the maximum difference in the sampled set, we obtain:

$$\|f(x) - g(x)\| \leq L_f\|x - x_i\| + M + L_g\|x - x_i\| = (L_f + L_g)\|x - x_i\| + M.$$

Now suppose that

$$\|x - x_i\| \leq \frac{\delta - M}{L_f + L_g},$$

possible as the quantity $\delta - M$ is positive. Then:

$$(L_f + L_g)\|x - x_i\| + M \leq (L_f + L_g) \frac{\delta - M}{L_f + L_g} + M = \delta - M + M = \delta.$$

Therefore,

$$\|f(x) - g(x)\| \leq \delta.$$

Since $x \in \mathcal{X}$ was arbitrary, the claim follows. □

Corollary 6.4.2 (Error Bound from an r -Cover Sampling Set). *Let $\mathcal{X} \subset \mathbb{R}^n$ and let $f, g : \mathcal{X} \rightarrow \mathbb{R}^m$.*

Assume f is L_f -Lipschitz and g is L_g -Lipschitz on $\mathcal{X}$, and that for a sampled set $\{x_i\}_{i=1}^N \subset \mathcal{X}$ we have

$$\|f(x_i) - g(x_i)\| \leq M \quad \text{for all } i.$$

Let $\delta > M$ and define

$$r := \frac{\delta - M}{L_f + L_g}.$$

If $\{x_i\}_{i=1}^N$ is an r -cover of $\mathcal{X}$, then

$$\sup_{x \in \mathcal{X}} \|f(x) - g(x)\| \leq \delta.$$

Proof. Let $x \in \mathcal{X}$ be arbitrary. Since $\{x_i\}_{i=1}^N$ is an r -cover of $\mathcal{X}$, by definition there exists some sampled point x_i such that

$$\|x - x_i\| \leq r.$$

By the choice of r ,

$$\|x - x_i\| \leq \frac{\delta - M}{L_f + L_g}.$$

All assumptions of Theorem 6.4.1 are satisfied, so it can be applied to conclude that

$$\|f(x) - g(x)\| \leq \delta.$$

Since $x \in \mathcal{X}$ was arbitrary, it follows that

$$\sup_{x \in \mathcal{X}} \|f(x) - g(x)\| \leq \delta.$$

□

This corollary is the key to exploit Lipschitz continuity to extend the certificate guarantees onto the true dynamic system through sampling density. From now on, we will refer to the errors on the dynamic with respect to the vector ℓ_∞ norm, as it allows for a simple grid-like structure for the sampling space. Additionally, in order to verify robustness with Marabou, only the maximum constraint is supported, while the non-linearities of other standard norms like the ℓ_2 norm are not supported.

To ensure that our sampling set satisfies the r -cover properties in Corollary 6.4.2, we propose a Uniform Cartesian Grid structure, where each point is separated by a length h in each dimension of the state space. More rigorously, given a domain $\mathcal{X} \subset \mathbb{R}^n$, the sampling space $\mathcal{S}$ is defined as:

$$\mathcal{S} = \{x \in \mathcal{X} \mid x = (k_1 h, \dots, k_n h), k_i \in \mathbb{Z}\}.$$

Proposition 6.4.3 (Grid Covering Radius in ℓ_∞). *Let $h > 0$ and let*

$$\mathcal{G} = \{(k_1 h, \dots, k_n h) \mid k_i \in \mathbb{Z}\} \subset \mathbb{R}^n$$

be the Uniform Cartesian Grid with spacing h .

Then for every $x \in \mathbb{R}^n$,

$$\min_{x_i \in \mathcal{G}} \|x - x_i\|_\infty \leq \frac{h}{2}.$$

Proof. For each coordinate x_j , choose $k_j \in \mathbb{Z}$ such that $|x_j - k_j h| \leq h/2$. Let $x_i = (k_1 h, \dots, k_n h) \in \mathcal{G}$. Then $\|x - x_i\|_\infty = \max_j |x_j - (x_i)_j| \leq h/2$. $\square$

Given by this proposition, the sampling space $\mathcal{S}$ defined above would entail an $\frac{h}{2}$ -cover of our domain $\mathcal{X}$. Moreover, this bound is tight as equality holds when x is located exactly at the center of a grid cell.

Combining this proposition with Corollary 6.4.2, we obtain the following crucial theorem.

Theorem 6.4.4 (Sampling Space for Desired Error Bound). *Let $\mathcal{X} \subset \mathbb{R}^n$ be a domain, and let $f, g : \mathcal{X} \rightarrow \mathbb{R}^m$ be L_f - and L_g -Lipschitz, respectively, with respect to the ℓ_∞ norm.*

Assume there exists a Uniform Cartesian Grid $\mathcal{G} \subset \mathcal{X}$ with spacing $h > 0$ such that the sampling error is bounded by:

$$\max_{x_i \in \mathcal{G}} \|f(x_i) - g(x_i)\| \leq M,$$

for some $M \geq 0$. Let $\delta > 0$ with $\delta > M$. If the grid spacing satisfies

$$h \leq \frac{2(\delta - M)}{L_f + L_g},$$

then the difference between f and g at any point over the whole domain is uniformly bounded above by δ :

$$\sup_{x \in X} \|f(x) - g(x)\| \leq \delta.$$

Proof. Let

$$r := \frac{\delta - M}{L_f + L_g}.$$

Since $\delta > M$, we have $r > 0$. The assumption

$$h \leq \frac{2(\delta - M)}{L_f + L_g}$$

implies

$$\frac{h}{2} \leq \frac{\delta - M}{L_f + L_g} = r.$$

Now fix $x \in \mathcal{X}$. Because $\mathcal{G}$ is a Uniform Cartesian Grid with spacing h , Proposition 6.4.3 implies that there exists some grid point $x_i \in \mathcal{G}$ such that

$$\|x - x_i\|_\infty \leq \frac{h}{2} \leq r.$$

so $\mathcal{G}$ is an r -cover of $\mathcal{X}$ with respect to the ℓ_∞ norm.

Since f and g are L_f - and L_g -Lipschitz on $\mathcal{X}$ respectively, and the sampling error is bounded by M , all assumptions of 6.4.2 are satisfied. Therefore,

$$\sup_{x \in \mathcal{X}} \|f(x) - g(x)\| \leq \delta.$$

This proves the claim. □

This theorem gives a concrete answer to how densely we must sample our state space when training our learned dynamic if our objective is to bound the approximation error over the true dynamic by δ . We formulate this Uniform Cartesian Grid for each case study and evaluate how feasible training in such circumstances would be.

Given the requirements for a robust Lyapunov Barrier certificate as expressed in 5.15, the descent parameter that we utilize during the CEGIS pipeline must be greater than the product of its Lipschitz constant and the perturbation bound on the next state. In our case, the only source of error in the evaluation of the next state will be the error of our surrogate dynamic. This means we can coordinate the maximum error bound of our dynamic with a bound on the certificate Lipschitz to ensure that the robust certificate is valid. Evidently, this new lower bound on the descent parameter is a strong constraint and might pose difficulties when converging to a working certificate-controller pair with our approach.

6.5 Empirical Experiment

Apart from attempting to achieve the conditions described in the previous section to transfer guarantees, we also evaluate the practical performance of the models trained using the learned dynamic. Concretely, we simulate N trajectories for each of our controllers that begin somewhere in the safe region of the domain. To be exact, we sample N initial points that satisfy $V(x) < \alpha$, where α is the safe threshold of the certificate, for which we expect a trajectory that will converge to the goal region and will never reach an unsafe state. Thus, we empirically show that our trained controller always satisfies the liveness and safety properties in the desired region. The average, standard deviation, and maximum steps taken for each controller to dock are recorded to compare the speed at which the finetuned controller docks with that of the initial controller.

Chapter 7

Case Studies

7.1 Two Dimensional Docking

In this benchmark, a spacecraft is trained using deep reinforcement learning in a two dimensional setting. The agent attempts to navigate the environment to eventually reach a docking region while avoiding unsafe areas and satisfying a nonlinear distance dependent velocity constraint. The controller maneuvers the spacecraft by providing forces in the x and y directions. The case study is adapted from Ravaioli et al. [65] and represents a *deputy* spacecraft that attempts to dock inside a radius surrounding the *chief* spacecraft. A diagram of this benchmark can be seen in Figure 7.1. We work with this benchmark as it includes evident safety and liveness concerns, and it has been proposed as a challenge in the field of control to solve the RWA task at hand with formal guarantees.

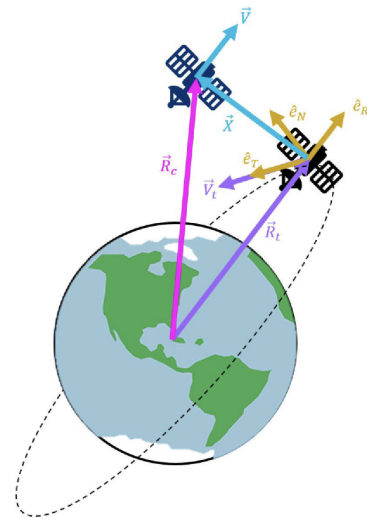


Figure 7.1: The 2D Docking scenario (adapted from [65]).

7.1.1 System Dynamics

The form of the states is the following: $[x, y, \dot{x}, \dot{y}]^T$, where (x, y) is the position, and $(\dot{x}, \dot{y})$ are the velocities. The control inputs are $[F_x, F_y]$, where each input is the thrust force applied to each direction, respectively. Forces are allowed to range from -1 to 1 , which is enforced in the controller wrapper by using piecewise linear clipping. The general transition model for this benchmark uses the Clohessy-Wiltshire orbital motion linear approximation in the non-inertial Hill's reference frame [25], [41]. We take the model constants as in the original scenario, where the *deputy* spacecraft's mass, m , is 12kg, gravity, g , is 9.81, and the *chief* spacecraft is located at the origin. The Ordinary Differential Equations that model the continuous time system dynamics are the following:

$$\dot{\mathbf{x}} = [\dot{x}, \dot{y}, \ddot{x}, \ddot{y}]^T \quad (7.1)$$

$$\ddot{x} = 2n\dot{y} + 3n^2x + \frac{F_x}{m} \quad (7.2)$$

$$\ddot{y} = -2n\dot{x} + \frac{F_y}{m} \quad (7.3)$$

where $n = 0.0010127$ rad/s.

However, the certificates for reach-while-avoid tasks are for discrete time dynamics, and so we opt for a closed-form discrete time solution for time-step T to this system as used in Mandal et al. [56], obtained by numerically integrating the described ODE:

$$\mathbf{x}(t_i + T) = \mathbf{x}(t_i) + \int_{t_i}^{t_i+T} \dot{\mathbf{x}}(\tau) d\tau \quad (7.4)$$

The resulting closed-form solution to obtain successive states is the following:

$$\begin{aligned} x' = & \left(\frac{2v_y}{n} + 4x + \frac{F_x}{mn^2} \right) + \left(\frac{2F_y}{mn} \right) \\ & + \left(-\frac{F_x}{mn^2} - \frac{2v_y}{n} - 3x \right) \cos(nT) + \left(-\frac{2F_y}{mn^2} + \frac{v_x}{n} \right) \sin(nT) \end{aligned} \quad (7.5)$$

$$y' = \left(-\frac{2v_x}{n} + y + \frac{4F_y}{mn^2}\right) + \left(-\frac{2F_x}{mn} - 3v_y - 6nx\right)T - \frac{3F_y}{2m}t^2 \\ + \left(-\frac{4F_y}{mn^2} + \frac{2v_x}{n}\right)\cos(nT) + \left(\frac{2F_x}{mn^2} + \frac{4v_y}{n} + 6x\right)\sin(nT) \quad (7.6)$$

$$v'_x = \left(\frac{2F_x}{mn}\right) + \left(-\frac{2F_y}{mn} + x\right)\cos(nT) + \left(\frac{F_x}{mn} + 2v_y + 3nx\right)\sin(nT) \quad (7.7)$$

$$v'_y = \left(-\frac{2F_x}{mn} - 3v_y - 6nx\right) + \left(-\frac{3F_y}{m}\right)T \\ + \left(\frac{2F_x}{mn} + 4v_y + 6nx\right)\cos(nT) + \left(\frac{4F_y}{mn} - 2v_x\right)\sin(nT) \quad (7.8)$$

7.1.2 Constraints and Conditions

The state space is defined in $\mathcal{X} = \mathbb{R}^4$, with the safe region being constrained to $\mathcal{X}_{safe} = [-2, 2]^2 \times [-0.5, 0.5]^2$, and the unsafe region being its complement $\mathcal{X}_{\mathcal{U}} = \mathcal{X} \setminus \mathcal{X}_{safe}$. The goal region is $\mathcal{X}_{\mathcal{G}} = [-0.35, 0.35]^2 \times \mathbb{R}^2$ and the initial space is $\mathcal{X}_{\mathcal{I}} = [-1, 1]^2 \times [0, 0]^2$. The original benchmark Ravaioli et al. [65] defines the goal region as a circle centered about the origin $(0, 0)$ of diameter $d = 1\text{m}$. We use this version for training, but due to nonlinearity, we verify on a square of side length $l = 0.7\text{m}$ so that we can encode it with linear inequalities.

The deputy spacecraft is additionally constrained on its velocity in order to maintain safety. This constraint is dependent on the distance from the chief spacecraft, and is imposed on the maximal magnitude of the velocity vector:

$$\sqrt{\dot{x}^2 + \dot{y}^2} \leq 0.2 + 2n\sqrt{x^2 + y^2} \quad (7.9)$$

Since this constraint is non-linear, it is not supported by neural network verification engines. For this reason, we adopt a linear over-approximation using OVERT, which ensures validity under the true constraint [67]. This velocity constraint is incorporated into training by augmenting the unsafe set $\mathcal{X}_{\mathcal{U}}$ with the states that violate the condition.

7.1.3 Experimental Setup

All experiments for this case study were conducted on a 14-inch Apple MacBook Pro equipped with an Apple M4 chip and 16 GB of memory, running macOS Sequoia 15.6.1. Verification queries were proved using the *Marabou* neural network verification engine [48, 78].

For the structure of the neural surrogate dynamic, we opt for a small network with one hidden layer of 64 neurons. The DNN is trained for 50 epochs with a Mean Squared Loss (MSE) loss criterion, the Adam Optimizer, and a learning rate of 10^{-3} , decaying by a factor of 10 every fifth epoch. We train on the dataset by loading the sampled points in batches of 64 in random order. These hyper-parameters were chosen through ample experimentation to optimize performance.

Following the structures presented in Mandal et al. [56], we construct a DNN structure of two hidden layers with 20 neurons for the controller policy, and one of two hidden layers with 30 neurons for the certificate. We include ReLU activations for each layer in both networks. Before the CEGIS finetuning process, we load an initial controller trained using a Proximal Policy Optimization (PPO) RL algorithm with the Ray RLLib library by Mandal et al. [56].

During the CEGIS loop, we use the following hyper parameters: $\beta = 1$, $\alpha = 1 + 10^{-5}$, $\epsilon = 5 \times 10^{-6}$ based in the original implementation [56]. The filter wrapper for the RWA certificate is defined as $V(x) = -10$ for all $x \in \mathcal{X}_G$, $V(x) = 1.2$ for all $x \in \mathcal{X}_U$, and $V(x) = N(x)$ for all other x where N is the neural representation of the certificate. For the weighting parameters, we choose $c_s = c_l = 1$ and $c_d = 10$, and for the over-adherence parameters, we use $\delta_1 = 10^{-4}$ and $\delta_2 = \delta_3 = 0$. The reason for the higher weight on the descent condition is that constraint 5.9 is harder to satisfy than constraint 5.8, and so we veer the training towards the harder objective. We additionally experiment with the loss term bounding the Lipschitz constant 6.4, for which we use an objective bound of $\tau = 1$. We progressively decreased this number experimentally and were allowed a low upper bound due to the simplicity of the benchmark. We use a learning rate of 5×10^{-3} when training the controller and certificate pair on the initial set of data. Once counterexamples are introduced, we finetune on a learning rate of 10^{-4} . In the loop, we train until a loss of 0 is achieved and then proceed with the verification. If counterexamples arise, we repeat the process until it is fully verified.

7.1.4 Results

The transition of the 2D Docking RWA solution from Mandal et al. [56] to one with a learned neural network dynamic proved to be satisfactory.

The dynamic model was trained in 134 seconds, obtaining an MSE of 2.8×10^{-14} in the training set. Similarly, we record a maximum ℓ_∞ -error $M = 1.9 \times 10^{-6}$ over all samples. We additionally calculate a Lipschitz bound for the dynamic of $L_2 \leq 2.51$ using equation 5.16, and based on the norm conversion formula 5.17, we obtain $L_\infty \leq K_{\infty,2} \cdot 2.51 = \frac{\sqrt{6}}{\sqrt{4}} \cdot 2.51 = 3.07$.

Then, we proceed to run the CEGIS workflow as explained in Section 6.3. The runtime of each training and verification iteration, as well as the number of counterexamples, are displayed in Table 7.1.

Iteration	Training Time (s)	Verification Time (s)	# Counterexamples
0	2101	292	3
1	347	290	6
2	158	279	2
3	234	299	0

Table 7.1: CEGIS iteration statistics.

We are able to synthesize a controller-certificate pair for the 2D Docking RWA task with the non-linear safety velocity constraint in four iterations. As expected, most of the training time is spent during the first iteration, where the certificate is initialized and trained to satisfy the conditions on most of the training set. The total time to solve the task was 2840s for training, 1160s for verification, and a grand total of 4000s, which is 66m and 40s.

In particular, we obtain a valid certificate for the task under the discrete dynamic transition function $\hat{f}$, but we want to extend the guarantee to the true system f . Due to training objective 6.4, we know that $L_\infty \leq 1.0$ for the certificate V . Substituting into 5.15, we obtain that $\epsilon_r > \delta$ must hold in order to validate the certificate on the true dynamic. That is, the ℓ_∞ -error of $\hat{f}$ must be bounded above by the decrease condition parameter $\epsilon_r = 5 \times 10^{-6}$. Applying Theorem 6.4.4, we would require a Uniform Cartesian Grid with diameter:

$$h \leq \frac{2(\delta - M)}{L_{\hat{f}} + L_f} = \frac{2(5 \times 10^{-6} - (1.9 \times 10^{-6}))}{3.07 + 3.5} = 9.44 \times 10^{-7},$$

where we use a conservative estimate that the Lipschitz constant of the true dynamic f is at most 3.5. This estimate is based on the bound for our accurate neural model, with an extra margin to be safe. Thus, the numbers that we provide represent a *conditional* guarantee on the spacing required for the grid. Based on the state space definition, we can calculate the amount of points necessary in the Uniform Cartesian Grid $\mathcal{G}$. For each of the six dimensions, we want to divide the range of the values, i.e. $[-2, 2]$ for the positions, by the total spacing, to get the number of nodes per lattice dimension, and then multiply these nodes to obtain all possible combinations of nodes for all dimensions:

$$\left(\frac{2 - (-2)}{h}\right)^2 \cdot \left(\frac{1.5 - (-1.5)}{h}\right)^2 \cdot \left(\frac{1 - (-1)}{h}\right)^2 \approx 8.1 \times 10^{38}.$$

This becomes an important drawback, since training a neural network on this amount of points is not feasible. Even by altering certain parameters, by obtaining a valid certificate with a larger descent epsilon such as $\epsilon = 10^{-3}$, or getting a lower bound on the Lipschitz constants to loosen the grid spacing requirements, the high dimensionality of the dynamic (6) and the size of the domain require a massive lattice to fully cover the space. For this reason, it seems that formally bounding the error of the learned dynamic through Lipschitz theory will not be possible.

Despite this, the error bound theorem 6.4.4 takes into account the worst possible behavior for the surrogate dynamic between sampled points. That is, when moving in any direction vector $\vec{x}$ in the dynamic input space, the dynamic model $\hat{f}$ will steer away from the true prediction by f by the amount L_∞ , as it is its maximum slope in the domain. However, there is no reason to believe that the trained dynamic will act in this sporadic manner outside of the sampled set, and the areas where its slope are the highest are most probably learned from areas of greater change in the true dynamic system.

Furthermore, we verify the controller obtained at different levels of robustness, as encoded in the query 6.7. After incrementing the perturbation radius until verification failed, we observed that the certificate is valid for a radius up to 10^{-4} . Because the maximal error of the dynamic is $M = 1.9 \times 10^{-6}$, which is two orders of magnitude lower than the robustness guarantee, it is sensible to predict that even if the maximum error over the entire domain is greater than M , it is most probably not going to exceed the limits of the guarantees provided by the certificate.

Additionally, the controller trained with this framework is empirically tested as described in Section 6.5. We sample 5,000 random points from the *safe* region of the domain $\{x \in \mathcal{X} \setminus \mathcal{X}_G \mid V(x) < \beta\}$, which is slightly larger than the initial set $\mathcal{X}_I$. We then simulate trajectories starting from these points by evaluating the initial controller and the trained controller in each state and prompting the black-box true dynamic f . Results can be seen in Table 7.2, and show that the approach on the surrogate dynamic remains competitive to training on the true system. In terms of liveness, both controllers were able to dock in the target region before a timeout of 2000 time steps was reached on all trajectories. The initial controller did not always satisfy the safety constraints: 549 trajectories were deemed unsafe. In contrast, the trained controller maintained safe behavior in all trajectories as expected. It is worth remarking that the safety violations on the initial controller were all due to the nonlinear velocity constraint despite being trained to adhere to it. The avoidance of the unsafe region proved to be an easier safety constraint, as both controllers satisfied it. Finally, we note that the initial controller tends to dock faster, doing so in an average of 6.07 steps, versus 15.40 and 10.66 steps for the certified controllers. Intuitively, training the controller to satisfy the certificate and velocity constraints makes the policy more conservative.

Controller	# Docks	# Unsafe	Avg. Steps	Max Steps
Initial	5,000	549	6.07 ± 1.81	11
Trained (True)	5,000	0	15.40 ± 7.25	28
Trained (Surrogate)	5,000	0	10.66 ± 3.99	23

Table 7.2: Controller Performance across 5,000 Trajectories.

7.2 Inverted Pendulum

This benchmark simulates an unstable system, where a weight is balanced above a pivot point and requires continuous control from a torque applied at the base to keep it upright. Here, we want to design a controller that keeps the vertical angle small while maintaining near-zero velocity and preventing it from falling down. This is a key benchmark in control theory for testing controllers with stabilization objectives. The case study is taken directly from Open AI Gym [17]. The controller’s job is to apply a torque based on the current pendulum angle to push rotation left or right as needed. This scenario is visualized in Figure 7.2.

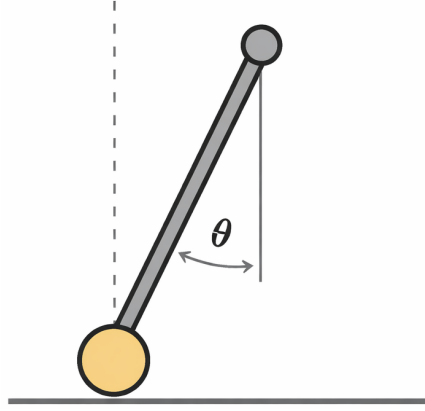


Figure 7.2: The inverted pendulum scenario.

7.2.1 System Dynamics and Constraints

The elements of the state space are smaller than in the 2D Docking benchmark; they are now comprised of two dimensions: $[\theta, \dot{\theta}]^T$, which are the vertical angle and the angular velocity respectively. The applied torque is a single input $u \in [-1, 1]$, which is given by the policy $\pi(x)$ and then run through piecewise linear clipping. The discrete time-step closed loop transitions for the dynamic system are defined:

$$\dot{\theta}' = (1 - b)\dot{\theta} + \left(\frac{1.5g \sin \theta}{2l} + \frac{3}{ml^2} \cdot 2u \right) T \quad (7.10)$$

$$\theta' = \theta + \dot{\theta}' T \quad (7.11)$$

where $g = 10\text{m/s}^2$, $m = 0.15\text{kg}$, $l = 0.5\text{m}$, $b = 0.1$, and $T = 0.05\text{s}$.

We take the same state sets as in the previous work [73], with $\mathcal{X} = [-0.7, 0.7]^2$, $\mathcal{X}_I = [-0.3, 0.3]^2$, $\mathcal{X}_G = [-0.2, 0.2]^2$, $\mathcal{X}_U = [-0.7, -0.6] \times [-0.7, 0] \cup [0.6, 0.7] \times [0, 0.7]$.

In this case study, we are not working with any difficult non-linear constraints on angular velocity. However, the dynamic transition function is now non-linear as it evaluates the trigonometric function $\sin \theta$ on one of the input variables, which makes controller synthesis harder. This benchmark is widely used to evaluate control and verification methods due to its low dimensionality combined with inherently unstable and nonlinear dynamics.

7.2.2 Experimental Setup

The same machine, training, and verification frameworks as in the 2D Docking benchmark were used.

We design a network with three hidden layers of 64 neurons for the structure of the neural surrogate dynamic. The DNN is trained for 140 epochs with a Mean Squared Loss (MSE) loss criterion and the Adam Optimizer. Training begins with a learning rate of 10^{-3} , which we decrease twice by a factor of 10 at epochs 50 and 100. To reduce maximum loss, we also include a second phase of training each epoch that focuses on the top- k samples with the greatest errors. We choose k to be 3% of the points, perform two tail passes per epoch, and have a weight of 0.5 on the tail loss for every epoch. The hyper-parameters were tuned to optimize performance and balance between low MSE and low maximal error.

For the controller policy, we use a DNN structure of two hidden layers with 128 neurons, while the certificate is represented by a network with three hidden layers with 64, 32 and 16 neurons, respectively, as done in previous work [73, 90]. All layers are separated by ReLU activation functions. Similarly, we start from a pretrained PPO-based RL controller before starting the CEGIS loop [73].

During controller and certificate synthesis, we use the following hyperparameters: $\beta = 1$, $\alpha = 1 + 10^{-5}$, $\epsilon = 5 \times 10^{-6}$ based on the original implementation [73]. The filtered wrapper for the RWA certificate was defined in the same way as for the 2D docking case, with unsafe states being assigned 1.2, goal states being -10 , and the neural network evaluated at the state otherwise. For the weighting parameters, we utilize a schedule that focuses on fixing the safe constraint 5.8 first, and then on the descent condition 5.9. For the first five epochs we set $c_s = 10$, and $c_d = 1$, and after we change the parameters to $c_s = 1$, and $c_d = 10$. For the over-adherence parameters, we set $\delta_1 = 10^{-5}$ and $\delta_2 = 0$. We additionally experiment with the loss term bounding the Lipschitz constant 6.4, for which we use an objective bound of $\tau = 3$, higher than in the last case-study, as the lower Lipschitz bound would not allow for convergence. We use a learning rate of 10^{-3} when training the controller and certificate pair on the initial set of data, and a learning rate of 10^{-4} for finetuning on the counterexamples. We follow the same process here where we train until a loss of 0 is achieved and then proceed with the verification, repeating until our certificate on the controller is valid.

7.2.3 Results

For this benchmark, we consistently obtain valid certificates in relation to the dynamic in 1 or 2 epochs. This suggests that the initial controller trained simply with RL techniques is already of good quality, and also it is simpler to comply with certificate requirements as there are no complex non-linear constraints to follow like in the 2D-Docking benchmark.

The total training time of the dynamic was 745 seconds, achieving an MSE of 9.86×10^{-7} in the training set. The maximum ℓ_∞ -error on the sample set was $M = 0.0057$. Similarly, we utilize equations 5.16 and 5.17 to obtain an upper bound on the Lipschitz constant using the spectral norm of the network, yielding 946.03.

The total time for the CEGIS loop to converge to a valid certificate was 1337s or 22m and 17s, with 1317s coming from training and 20s from verification. Our controller was synthesized in a single iteration, where after training, no counterexamples were found.

Iteration	Train Time (s)	Verify Time (s)	# Counterexamples
0	1317	20	0

Table 7.3: CEGIS iteration statistics.

We also experimented with the Lipschitz loss term 6.4. When training with the true dynamic, we were able to replicate the work in Wang et al. [73], and obtain a certificate with a constant bounded by 3. However, due to the irregularities of training under a neural model transition function, it was harder to attain 0 training loss with this objective function. We tested various upper bounds for the loss term, and were able to converge training for a bound as low as $\tau = 7$. It is also important to note that when we constrain the certificate by the value of its spectral norm, training becomes more irregular; some trials could reach 0 loss in 50 epochs and take 3 verification iterations, or it could take as much as 200 epochs and a single CEGIS iteration.

While our certificate Lipschitz bound was a tight 7, Equation 5.15 implies that we need our transition perturbations to be bounded as such: $\frac{\epsilon_r}{L_\infty} > \delta$. Substituting by the numbers used, we get $\frac{5 \times 10^{-6}}{7} > \delta$. However, the maximum error on the sample set is $M = 0.0057 > \frac{5 \times 10^{-6}}{7}$. Thus, we will not be able to translate the certificate guarantees on $\hat{f}$ to the true system f with this lemma. In a black box access

situation like this one, it would be highly unlikely to be able to obtain a neural model as accurate as required for a nonlinear system like the inverted pendulum. Moreover, the verifying descent parameter ϵ_r could also be increased to help with the inequality condition, but then the CEGIS convergence would be hindered. As the inequality is not met, the formula in Theorem 6.4.4 does not apply and so we cannot calculate the spacing needed for a cover on the domain, or calculate the number of points needed to sample. It is interesting to note that if the inequality was satisfied, the number of points required would be much lower, as now we have 3 input dimensions on $\hat{f}$ instead of the 6 in 2D Docking.

Despite the failure to translate the guarantees to the true system, we analyze the robustness of the certificate with query 6.7. We were able to fully verify our certificate up to a perturbation radius of 9×10^{-3} . This number is still larger than the maximum error we obtained in the samples, $M = 0.0057$, but by a much smaller gap than in the 2D Docking scenario. Although this is a good sign, there is a greater chance that the errors for the surrogate dynamic exceed the bound of our guarantee globally.

We perform the same empirical test on 5,000 simulated trajectories starting in states in the following set: $\{x \in \mathcal{X} \setminus \mathcal{X}_G \mid V(x) < \beta\}$, which is a superset of $\mathcal{X}_I$. We compare the performance of the initial controller and the controllers trained on the real and the surrogate dynamics in satisfying the liveness and safety properties. The results show that all controllers were able to dock before timeout, results parallel to the previous scenario. In terms of safety, even the initial controller was able to satisfy the constraints. However, the only condition we had was not to enter the *unsafe* region, which is defined as extreme angles with velocity in the same direction, leading the pendulum out of the state space. This constraint is much simpler for the controller to adhere to than the nonlinear velocity constraint in 2D Docking 7.9. Once again, the initial controller docks in fewer steps than both trained controllers, doing so in an average of 9.68 steps, while the other controllers dock in 11.25 and 57.29.

Controller	# Docks	# Unsafe	Avg. Steps	Max Steps
Initial	5,000	0	9.68 ± 0.77	11
Trained (True)	5,000	0	11.25 ± 4.06	22
Trained (Surrogate)	5,000	0	57.29 ± 24.52	94

Table 7.4: Controller Performance across 5,000 Trajectories.

The results of the trajectory simulations reflect that the surrogate dynamic performs similarly to its

counterparts. The initial controller proved to be up to standard, since it obtained no errors in safety or liveness in the region of interest. This means that we do not have a true point of comparison for the true trained models, which performed equally as well. Nevertheless, we know that the training process did not lead us astray and all of its guarantees still held empirically. An important thing to comment is that with training on the surrogate model, the controller did become more conservative, and it is taking more time steps to dock than the other two controllers.

7.3 Two Dimensional Quadrotor

The two dimensional quadrotor is a device that has two rotors with propellers that provide an upward force for vertical takeoff and control. The goal of this benchmark is to hover at an equilibrium position, which requires the right amount of vertical force to counteract the weight of the quadrotor. We aim to produce a controller that achieves such hovering while avoiding large tilts and getting too far away from equilibrium. We include this case study because it is known in the control community as a very complex benchmark [64], [59], [81]. It was not until recently that a formal certificate was made for a 2D Quadrotor control task. The quadrotor system can be seen in Figure 7.3.

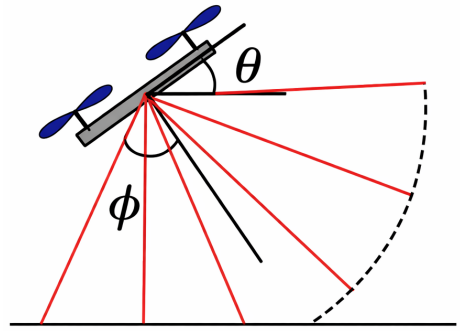


Figure 7.3: The 2D Quadrotor scenario.

7.3.1 System Dynamics and Constraints

The state space of this benchmarks includes elements of 6 dimensions of the form: $[x, z, \theta, \dot{x}, \dot{z}, \dot{\theta}]^T$, where (x, z) are the horizontal and vertical positions, $(\dot{x}, \dot{z})$ are the velocities in the same directions, θ is the vertical tilt and $\dot{\theta}$ is the angular velocity. The main objective of the system is to stabilize at an equilibrium position with velocities and angles, so the conceptual goal state is defined as $x^* = \mathbf{0}$. The controller input is defined as the force applied through each rotor on each side of the quadrotor, in the form of $u = [u_1, u_2]^T$, $u \in \mathbb{R}_{\geq 0}^2$. The forces must adhere to the following maximal constraint: $\|u\|_{\infty} \leq 1.25mg$. Once again, the

dynamic transition function is defined with a set of ordinary differential equations:

$$\ddot{x} = -\frac{1}{m} \sin(\theta)(u_1 + u_2) \quad (7.12)$$

$$\ddot{y} = \frac{1}{m} \cos(\theta)(u_1 + u_2) - g \quad (7.13)$$

$$\ddot{\theta} = \frac{l}{I}(u_1 - u_2), \quad (7.14)$$

where $m = 0.486$, $l = 0.25$, $I = 0.00383$, and $g = 9.81$ [81].

The relevant state space is defined by a scalar multiple $p = 0.8$ of the bounds $b = [0.75, 0.75, \frac{\pi}{2}, 4, 4, 3]$, so that $\mathcal{X} = \prod_{i=1}^6 [-pb_i, pb_i]$. The goal region is defined as a scaled subset of the state space, $\mathcal{X}_G = 0.1 \mathcal{X}$, while the unsafe region corresponds to the outer portion of the state space, $\mathcal{X}_U = \mathcal{X} \setminus (0.9 \mathcal{X})$. We also define an initial set $\mathcal{X}_I = \{x \in \mathcal{X} \mid V(x) < \rho\}$, where ρ is the safety threshold determined by our experiments.

7.3.2 Experimental Setup

For the 2D Quadrotor surrogate dynamic neural network, we incorporate three hidden layers of 128 neurons each, with ReLU activations in each layer. We train for 100 epochs with a Mean Squared Loss (MSE) objective function and the Adam Optimizer. An initial learning rate of 10^{-3} is used, scheduled to decrease by a factor of 10 at epochs 30, 60, and 90. We similarly implement the tail loss phase of training after each epoch to reduce maximum loss, training on the top- k samples with the greatest errors. We let k be 3% of the points, perform two tail passes per epoch, and have a weight of 0.5 on the tail loss for every epoch. All hyper-parameters are chosen to perform well on the MSE and max error objectives.

After much experimentation, we were unable to train a valid controller-certificate pair for the 2D Quadrotor problem with the CEGIS approach described in Mandal et al. [56]. Many parameters were tuned, including trying to downsize the domain to obtain a certificate in a very small region, but the approach was not scalable enough for a more complex scenario like this one. During the first iteration of the controller and certificate training, we observe that obtaining 0 loss was unattainable in most trials mainly due to the descent loss term 6.3. Increasing the size of the controller and certificate networks would allow the loss to

decrease but never converge, and there are significant drawbacks in the verification runtime from doing so. We additionally attempted to verify an intermediate controller and certificate to test the process and learned that the verification process is also a significant bottleneck, since the more complex queries pose a greater challenge and require unfeasible amounts of time.

For this reason, we opt to use the framework established in Yang et al. [81] to synthesize a certificate, since it is more scalable. All experiments are run on the same machine. This pipeline shares some key similarities with the original approach, since it is also based on a CEGIS loop. The framework trains the controller, the certificate and optionally an observer. The pipeline supports both a neural network and a quadratic structure for the certificate function. The main difference of this approach is that it focuses on a region of attraction (ROA), $\mathcal{S} = \{x \in \mathcal{X} \mid V(x) < \rho\}$, with $\rho \in \mathbb{R}^+$, and includes an additional loss term to encourage the expansion of the ROA. This pipeline formally guarantees that the state $x \in \mathcal{S}$ remains in the ROA after one action taken by the controller in the dynamical system. Moreover, the system uses projected gradient descent (PGD) attacks to obtain counterexamples instead of expensive verification engines, and instead does a final formal verification process after controller synthesis. We parameterize the Lyapunov candidate as a quadratic function centered on the goal state $x^* = \mathbf{0}$:

$$V(x) = (x - x^*)^\top Q(x - x^*) = x^\top Qx,$$

where

$$Q = \epsilon I + R^\top R.$$

Here, R is a trainable matrix and $\epsilon \geq 0$ is a fixed scalar which we set to $\epsilon = 0.01$. By construction, $R^\top R$ is positive semidefinite, and thus Q is positive definite whenever $\epsilon > 0$. Therefore, $V(x)$ is positive definite with respect to x^* , namely,

$$V(x^*) = 0, \quad V(x) > 0 \text{ for all } x \neq x^*,$$

satisfying the lower bound assumption. For the controller, we use a neural network with two hidden layers of 64 neurons each. More details on the implementation of the ROA based controller synthesis approach

can be found in Yang et al. [81].

7.3.3 Results

The total training time of the dynamic was 436 seconds, achieving an MSE of 7.9×10^{-7} in the training set. The maximum ℓ_∞ -error on the sample set was $M = 0.0114$. Similarly, we utilize equations 5.16 and 5.17 to obtain an upper bound on the Lipschitz constant using the spectral norm of the network, which yields 49.67.

Running the pipeline process, we observe different behaviors when replacing the true system with our neural model. For the true system, the workflow results in a controller certificate pair in roughly 427s, and the value of ρ increases as the adversarial loss from the PGD attacks reaches 0. For this experiment, we obtain a ROA with $\rho = 1.28$. In contrast, the process seems to struggle to reach an adversarial loss of 0 during training when using the surrogate dynamic. However, the tool has a mechanism to keep adding counterexamples and reducing/expanding the ROA as needed to eventually reach a controller certificate pair. The workflow attempts to improve the controller until an epoch limit is reached. The total run time amounts to roughly 3 hours and 44 minutes. For this benchmark, we omit discussion of extending the guarantee via Lipschitz to the true environment, since the controller obtained is not formally verified on the learned system, yet it is still trained to encourage the liveness and safety properties through the Lyapunov framework. For this reason, we still observe a positive outcome in our empirical results.

Due to the different nature of our methodology for this benchmark, we perform three different empirical tests: (1) in a predefined initial region $\mathcal{X}_I = 0.6\mathcal{X}$, (2) inside the ROA of the controller trained on the surrogate dynamic $\hat{\mathcal{S}}$, and (3) inside the ROA of the controller trained on the true dynamic $\mathcal{S}$. For each of these experiments, we still simulate 5,000 trajectories

Controller	# Docks	# Unsafe	Avg. Steps	Max Steps
Initial	5,000	2,735	786.3 ± 203.5	1,179
Trained (True)	5,000	2,291	347.8 ± 125.7	642
Trained (Surrogate)	4,990	2,265	225.0 ± 70.6	389

Table 7.5: Controller Performance across 5,000 Trajectories on initial region $\mathcal{X}_I$.

Table 7.5 displays the results for the arbitrary initial region, $\mathcal{X}_I$, which was chosen to provide a wide

margin from the unsafe region. Formally, controllers are only verified to solve the task within their ROA, but it is worth evaluating a deployment region that may more accurately assimilate a situation in real life. Despite the gap between the boundaries of $\mathcal{X}_{\mathcal{I}}$ and $\mathcal{X}_{\mathcal{U}}$, we still see many trajectories which reached an unsafe state at some point for all controllers. Despite this, the two finetuned controllers performed better in around 500 trajectories by remaining safer, with the surrogate approach leading by 26. It is important to note that the surrogate approach was slightly less robust in satisfying the liveness property in this initial region, as it reached the timeout limit of 2,000 epochs for 10 trajectories. Contrary to the other benchmarks, we actually see that the initial controller is almost twice as slow as the trained ones. This leads to the hypothesis that the magnitude of the actions taken by the controller is mostly determined by the training approach, rather than by whether or not we replace the true system with a surrogate dynamic. The surrogate approach tended to dock the fastest overall, with less variance in the number of steps taken every time.

Controller	# Docks	# Unsafe	Avg. Steps	Max Steps	Stayed in ROA
Initial	5,000	22	473.1 ± 180.3	784	4994
Trained (True)	5,000	12	252.8 ± 111.1	493	12
Trained (Surrogate)	5,000	10	158.2 ± 49.5	276	5000

Table 7.6: Controller Performance across 5,000 Trajectories starting in the surrogate ROA $\hat{\mathcal{S}}$.

The performance of the controllers under the surrogate ROA $\hat{\mathcal{S}}$ can be seen in Table 7.6. In this experiment, all controllers were able to dock successfully within the designated time limit. Moreover, it is evident that sampling inside the ROA helps maintain safety for all controllers, even if they are meant to work on different ROAs. The counterexample guided controllers still outperformed in both safety and speed in patterns similar to those of the first experiment. As we would expect from the guarantees of the approach, all trajectories from the surrogate dynamic remained constrained to the ROA, while almost all of the true dynamic left the region at some point.

For the final experiment using the ROA of the true dynamic certificate, we observe that safety is almost ubiquitous. Even the initial controller had only one dangerous trajectory. This result could imply that the region obtained might be smaller or closer to the origin so that it is easier for all controllers to satisfy the properties, even without counterexample inducted training. All trajectories properly docked

Controller	# Docks	# Unsafe	Avg. Steps	Max Steps	Stayed in ROA
Initial	5,000	1	440.9 ± 184.7	801	4997
Trained (True)	5,000	0	242.7 ± 115.4	486	5,000
Trained (Surrogate)	5,000	0	148.3 ± 52.6	269	4985

Table 7.7: Controller Performance across 5,000 Trajectories starting in the true ROA $\mathcal{S}$.

for all controllers and the finetuned checkpoints remained faster. As promised by the training approach developed by Yang et al. [81], all trajectories for the true dynamic remained in the ROA, while mostly all remained for the other two controllers. It is interesting that the opposite case does not hold, as we saw that the controller trained on the true dynamic did not remain under the surrogate ROA $\hat{\mathcal{S}}$. A reason for this is possibly that since the properties were able to be fully met by all counterexamples in the true system case, it had to be a smaller and more constrained region, whereas the framework opted for a larger ROA for the surrogate dynamic case since the 0 loss was not met regardless. This makes it so that not all trajectories sampled from $\hat{\mathcal{S}}$ will help the controller of the true system, as it was not trained in the whole region. The safety results on the true ROA might suggest that it is smaller as well, since there were very few cases of the property being broken, even for the initial controller.

With these results, it is safe to say that for the quadrotor setting, replacing the original transition function with a surrogate neural network will not result in a fully verified controller, yet it will still give empirical improvements for safety and for stabilizing in the ROA which should lead it to satisfy liveness.

7.4 Discussion

In these case studies, we observed that replacing the true system with the learned surrogate dynamic in controller-certificate synthesis workflows does not pose significant drawbacks. Using the CEGIS pipeline, we achieved controller-certificate pairs in roughly one hour, which is comparable to the time that synthesis takes using the true dynamic.

For the two dimensional quadrotor, we observe that the CEGIS workflow is not sufficient. However, even when evaluating time steps with the true dynamic, this workflow is unable to generate a valid certificate unless a very strong controller and Lyapunov candidate are available before finetuning. This suggests

that the inability to train the quadrotor controller is not a consequence of replacing the dynamic, but rather due to other confounding factors.

Moving to the ROA expansion framework, we observe that the neural model decelerates the training process, since the adversarial loss cannot be fully minimized and optimization proceeds until timeout. The 2D quadrotor is a complex control benchmark, with nonlinear dynamics and a six-dimensional state space with two control inputs. This complexity may introduce impurities in the learned dynamics, which prevent certificate descent at every time step.

Although training is slow and full guarantees are not obtained, finetuning with certificate objectives still provides improvements in controller behavior. Empirical evaluations on the true system showed that satisfying the liveness property was not an issue for any controller, and while safety was more problematic, we still see an improvement comparable to controllers trained under the true dynamic. Overall, surrogate dynamics can be used in controller synthesis for simpler scenarios, and there is promising potential for scaling these methods to more complex systems. Additionally, depending on the workflow and system dynamics, the docking velocity varies; in some cases the controllers are more conservative, while in others they are more efficient.

These results suggest that applying this controller synthesis approach in environments with unknown or black-box dynamics improves reliability compared to standard reinforcement learning controllers.

Chapter 8

Conclusion & Future Work

In this thesis, we presented a neuro-symbolic framework for synthesizing certified neural controllers for dynamical systems with black-box access. Our approach combines surrogate model learning with a CEGIS-based controller and certificate synthesis pipeline, enabling the application of formal verification techniques in settings where the true system dynamics are unknown.

We evaluated this framework on three benchmark systems: 2D docking, inverted pendulum, and a 2D quadrotor. The results demonstrated that surrogate dynamics can be effectively incorporated into the synthesis pipeline for low-dimensional systems, achieving comparable performance while maintaining manageable verification complexity. However, for more complex systems such as the quadrotor, scalability and approximation error remain significant challenges.

A central question addressed in this study is what theoretical foundations are required to extend certificate guarantees to the true system. We leveraged Lipschitz continuity to derive conditions on the sampling density required to transfer guarantees from the surrogate to the true dynamics. In practice, these guarantees are limited by the difficulty of maintaining sufficiently low approximation error in the learned dynamics, especially for complex systems. Additionally, the resulting sampling requirements can become prohibitively large, and no existing computer would be able to train a model on a sample set of such magnitude. Despite this, the analysis provides a conservative bound, and there is reason to believe that the true error is closer to the observed maximum error on sampled points. Under this assumption, we provide a conditional guarantee on the validity of the certificates for the true system.

Due to the limitations of this work and the promising findings, there are several avenues for future research. One key limitation is the inability to formally bound the approximation error of the surrogate dynamics. Exploring improved neural architectures or training methods to reduce the maximum ℓ_∞ error would be an important step toward achieving formal guarantees.

Additionally, incorporating objectives that control the Lipschitz constant of the learned dynamics may improve robustness to perturbations. It would also be valuable to investigate more scalable verification approaches, such as compositional certificates [56], and evaluate their compatibility with surrogate dynamics.

Further exploration of the ROA expansion framework in simpler systems such as 2D docking and inverted pendulum may provide insight into achieving larger verified regions. Moreover, evaluating the surrogate approach on additional benchmarks, such as cart-pole or higher-dimensional quadrotor systems, would help assess generalizability.

Finally, since the Lipschitz-based guarantees are highly conservative, developing tighter analytical bounds that better reflect the behavior of the learned dynamics between sampled points remains an important open problem.

Bibliography

- [1] Alessandro Abate, Daniele Ahmed, Alec Edwards, Mirco Giacobbe, and Andrea Peruffo. Fossil: a software tool for the formal synthesis of lyapunov functions and barrier certificates using neural networks. In *Proceedings of the 24th International Conference on Hybrid Systems: Computation and Control*, HSCC '21, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450383394. doi: 10.1145/3447928.3456646. URL <https://doi.org/10.1145/3447928.3456646>.
- [2] Alessandro Abate, Daniele Ahmed, Mirco Giacobbe, and Andrea Peruffo. Formal synthesis of lyapunov neural networks. *IEEE Control Systems Letters*, 5(3):773–778, July 2021. ISSN 2475-1456. doi: 10.1109/lcsys.2020.3005328. URL <http://dx.doi.org/10.1109/LCSYS.2020.3005328>.
- [3] Alessandro Abate, Sergiy Bogomolov, Alec Edwards, Kostiantyn Potomkin, Sadegh Soudjani, and Paolo Zuliani. Safe reach set computation via neural barrier certificates, 2024. URL <https://arxiv.org/abs/2404.18813>.
- [4] Daniele Ahmed, Andrea Peruffo, and Alessandro Abate. *Automated and Sound Synthesis of Lyapunov Functions with SMT Solvers*, page 97–114. Springer International Publishing, 2020. ISBN 9783030451905. doi: 10.1007/978-3-030-45190-5_6. URL http://dx.doi.org/10.1007/978-3-030-45190-5_6.
- [5] Saiedeh Akbari, Cristian F. Nino, Omkar Sudhir Patil, and Warren E. Dixon. Lyapunov-based deep neural networks for adaptive control of stochastic nonlinear systems, 2024. URL <https://arxiv.org/abs/2412.21095>.
- [6] Hassan Almubarak, Manan Gandhi, Yuichiro Aoyama, Nader Sadegh, and Evangelos A. Theodorou.

- Gaussian process barrier states for safe trajectory optimization and control, 2022. URL <https://arxiv.org/abs/2212.00268>.
- [7] Aaron D. Ames, Xiangru Xu, Jessy W. Grizzle, and Paulo Tabuada. Control barrier function based quadratic programs for safety critical systems. *IEEE Transactions on Automatic Control*, 62(8): 3861–3876, August 2017. ISSN 1558-2523. doi: 10.1109/tac.2016.2638961. URL <http://dx.doi.org/10.1109/TAC.2016.2638961>.
- [8] Aaron D. Ames, Samuel Coogan, Magnus Egerstedt, Gennaro Notomista, Koushil Sreenath, and Paulo Tabuada. Control barrier functions: Theory and applications, 2019. URL <https://arxiv.org/abs/1903.11199>.
- [9] Guy Amir, Haoze Wu, Clark Barrett, and Guy Katz. An smt-based approach for verifying binarized neural networks, 2021. URL <https://arxiv.org/abs/2011.02948>.
- [10] Mahathi Anand, Abolfazl Lavaei, and Majid Zamani. Compositional synthesis of control barrier certificates for networks of stochastic systems against ω -regular specifications, 2022. URL <https://arxiv.org/abs/2103.02226>.
- [11] Somil Bansal, Mo Chen, Sylvia Herbert, and Claire J. Tomlin. Hamilton-jacobi reachability: A brief overview and recent advances, 2017. URL <https://arxiv.org/abs/1709.07523>.
- [12] Clark Barrett, Roberto Sebastiani, Sanjit A. Seshia, and Cesare Tinelli. Satisfiability modulo theories. In *Handbook of Satisfiability*, volume 185, pages 825–885. IOS Press, 2009.
- [13] G. Basile and G. Marro. Controlled and conditioned invariant subspaces in linear system theory. *J. Optim. Theory Appl.*, 3(5):306–315, May 1969. ISSN 0022-3239. doi: 10.1007/BF00931370. URL <https://doi.org/10.1007/BF00931370>.
- [14] Felix Berkenkamp, Riccardo Moriconi, Angela P. Schoellig, and Andreas Krause. Safe learning of regions of attraction for uncertain, nonlinear systems with gaussian processes. In *2016 IEEE 55th Conference on Decision and Control (CDC)*, page 4661–4666. IEEE, December 2016. doi: 10.1109/cdc.2016.7798979. URL <http://dx.doi.org/10.1109/CDC.2016.7798979>.

- [15] Ruxandra Bobiti and Mircea Lazar. Sampling-based verification of lyapunov's inequality for piecewise continuous nonlinear systems, 2016. URL <https://arxiv.org/abs/1609.00302>.
- [16] Nicholas M. Boffi, Stephen Tu, Nikolai Matni, Jean-Jacques E. Slotine, and Vikas Sindhwani. Learning stability certificates from data, 2020. URL <https://arxiv.org/abs/2008.05952>.
- [17] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym, 2016. URL <https://arxiv.org/abs/1606.01540>.
- [18] Lukas Brunke, Melissa Greeff, Adam W. Hall, Zhacong Yuan, Siqi Zhou, Jacopo Panerati, and Angela P. Schoellig. Safe learning in robotics: From learning-based control to safe reinforcement learning, 2021. URL <https://arxiv.org/abs/2108.06266>.
- [19] Fernando Castañeda, Jason J. Choi, Bike Zhang, Claire J. Tomlin, and Koushil Sreenath. Gaussian process-based min-norm stabilizing controller for control-affine systems with uncertain input effects and dynamics, 2021. URL <https://arxiv.org/abs/2011.07183>.
- [20] Ya-Chien Chang and Sicun Gao. Stabilizing neural control using self-learned almost lyapunov critics, 2021. URL <https://arxiv.org/abs/2107.04989>.
- [21] Ya-Chien Chang, Nima Roohi, and Sicun Gao. Neural lyapunov control, 2022. URL <https://arxiv.org/abs/2005.00611>.
- [22] Shaoru Chen, Mahyar Fazlyab, Manfred Morari, George J. Pappas, and Victor M. Preciado. Learning lyapunov functions for hybrid systems, 2020. URL <https://arxiv.org/abs/2012.12015>.
- [23] Jason Choi, Fernando Castañeda, Claire J. Tomlin, and Koushil Sreenath. Reinforcement learning for safety-critical control under model uncertainty, using control lyapunov functions and control barrier functions, 2020. URL <https://arxiv.org/abs/2004.07584>.
- [24] Yinlam Chow, Ofir Nachum, Edgar Duenez-Guzman, and Mohammad Ghavamzadeh. A lyapunov-based approach to safe reinforcement learning, 2018. URL <https://arxiv.org/abs/1805.07708>.
- [25] W. H. Clohessy. Terminal guidance system for satellite rendezvous. *Journal of the Aerospace Sciences*, 2012. URL <https://api.semanticscholar.org/CorpusID:58706939>.

- [26] Charles Dawson, Zengyi Qin, Sicun Gao, and Chuchu Fan. Safe nonlinear control using robust neural lyapunov-barrier functions, 2021. URL <https://arxiv.org/abs/2109.06697>.
- [27] Charles Dawson, Sicun Gao, and Chuchu Fan. Safe control with learned certificates: A survey of neural lyapunov, barrier, and contraction methods, 2022. URL <https://arxiv.org/abs/2202.11762>.
- [28] J. L. C. B. de Farias and W. M. Bessa. Intelligent control with artificial neural networks for automated insulin delivery systems. *Bioengineering*, 9(11):664, 2022. doi: 10.3390/bioengineering9110664.
- [29] Alec Edwards, Andrea Peruffo, and Alessandro Abate. Fossil 2.0: Formal certificate synthesis for the verification and control of dynamical models, 2024. URL <https://arxiv.org/abs/2311.09793>.
- [30] Alec Edwards, Andrea Peruffo, and Alessandro Abate. A general framework for verification and control of dynamical models via certificate synthesis, 2024. URL <https://arxiv.org/abs/2309.06090>.
- [31] Yizhak Yisrael Elboher, Justin Gottschlich, and Guy Katz. *An Abstraction-Based Framework for Neural Network Verification*, page 43–65. Springer International Publishing, 2020. ISBN 9783030532888. doi: 10.1007/978-3-030-53288-8_3. URL http://dx.doi.org/10.1007/978-3-030-53288-8_3.
- [32] Huan Zhang et al. α, β -crown: A fast and scalable neural network verifier. <https://github.com/Verified-Intelligence/alpha-beta-CROWN>, 2021.
- [33] Yanbo Fan, Siwei Lyu, Yiming Ying, and Bao-Gang Hu. Learning with average top-k loss, 2017. URL <https://arxiv.org/abs/1705.08826>.
- [34] Jaime F. Fisac, Mo Chen, Claire J. Tomlin, and S. Shankar Sastry. Reach-avoid problems with time-varying dynamics, targets and constraints, 2014. URL <https://arxiv.org/abs/1410.6445>.
- [35] Milan Ganai, Chiaki Hirayama, Ya-Chien Chang, and Sicun Gao. Learning stabilization control from observations by learning lyapunov-like proxy models, 2023. URL <https://arxiv.org/abs/2303.02215>.
- [36] Sicun Gao, Soonho Kong, and Edmund M. Clarke. drealm: An smt solver for nonlinear theories over the reals. In *International Conference on Automated Deduction (CADE)*, pages 208–214. Springer, 2013.

- [37] Shromona Ghosh, Yash Vardhan Pant, Hadi Ravanbakhsh, and Sanjit A. Seshia. Counterexample-guided synthesis of perception models and control, 2021. URL <https://arxiv.org/abs/1911.01523>.
- [38] Peter Giesl and Sigurdur Hafstein. Review on computational methods for lyapunov functions. *Discrete and Continuous Dynamical Systems - Series B*, 20(8):2291–2331, 2015. doi: 10.3934/dcdsb.2015.20.2291.
- [39] Wassim M. Haddad and VijaySekhar Chellaboina. *Nonlinear Dynamical Systems and Control: A Lyapunov-Based Approach*. Princeton University Press, Princeton, NJ, 2008.
- [40] Lukas Hewing, Alexander Liniger, and Melanie N. Zeilinger. Cautious nmpc with gaussian process dynamics for autonomous miniature race cars. In *2018 European Control Conference (ECC)*, page 1341–1348. IEEE, June 2018. doi: 10.23919/ecc.2018.8550162. URL <http://dx.doi.org/10.23919/ECC.2018.8550162>.
- [41] George William Hill. Researches in the lunar theory. *American Journal of Mathematics*, 1(1):5–26, 1878.
- [42] Kurt Hornik. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257, 1991. ISSN 0893-6080. doi: [https://doi.org/10.1016/0893-6080\(91\)90009-T](https://doi.org/10.1016/0893-6080(91)90009-T). URL <https://www.sciencedirect.com/science/article/pii/089360809190009T>.
- [43] Kai-Chieh Hsu*, Vicenç Rubies-Royo*, Claire Tomlin, and Jaime Fisac. Safety and liveness guarantees through reach-avoid reinforcement learning. In *Robotics: Science and Systems XVII*, RSS2021. Robotics: Science and Systems Foundation, July 2021. doi: 10.15607/rss.2021.xvii.077. URL <http://dx.doi.org/10.15607/RSS.2021.XVII.077>.
- [44] Tong Huang, Sicun Gao, and Le Xie. A neural lyapunov approach to transient stability assessment of power electronics-interfaced networked microgrids. *IEEE Transactions on Smart Grid*, 13(1), 10 2021. ISSN 1949-3053. doi: 10.1109/tsg.2021.3117889. URL <https://www.osti.gov/biblio/2427393>.
- [45] Zachary Jarvis-Wloszek, Ryan Feeley, Weehong Tan, Kunpeng Sun, and Andrew Packard. Some controls applications of sum of squares programming. *42nd IEEE International Conference on Decision and*

- Control (IEEE Cat. No.03CH37475)*, 5:4676–4681 Vol.5, 2003. URL <https://api.semanticscholar.org/CorpusID:18046057>.
- [46] Wanxin Jin, Zhaoran Wang, Zhuoran Yang, and Shaoshuai Mou. Neural certificates for safe control policies, 2020. URL <https://arxiv.org/abs/2006.08465>.
 - [47] Guy Katz, Clark Barrett, David Dill, Kyle Julian, and Mykel Kochenderfer. Reluplex: An efficient smt solver for verifying deep neural networks, 2017. URL <https://arxiv.org/abs/1702.01135>.
 - [48] Guy Katz, Derek A. Huang, Duligur Ibeling, Kyle Julian, Christopher Lazarus, Rachel Lim, Parth Shah, Shantanu Thakoor, Haoze Wu, Aleksandar Zeljic, David L. Dill, Mykel J. Kochenderfer, and Clark Barrett. The Marabou framework for verification and analysis of deep neural networks. In *cav*, 2019. doi: 10.1007/978-3-030-25540-4_26.
 - [49] Hassan K. Khalil. *Nonlinear Systems*. Prentice Hall, Upper Saddle River, NJ, 3 edition, 2002.
 - [50] Abolfazl Lavaei and Emilio Frazzoli. Scalable synthesis of safety barrier certificates for networks of stochastic switched systems. *IEEE Transactions on Automatic Control*, 69(11):7294–7309, 2024. doi: 10.1109/TAC.2024.3386633.
 - [51] Boqian Li, Shiping Wen, Zheng Yan, Guanghui Wen, and Tingwen Huang. A survey on the control lyapunov function and control barrier function for nonlinear-affine control systems, 2023. ISSN 2329-9266. URL <https://www.ieee-jas.net/en/article/doi/10.1109/JAS.2023.123075>.
 - [52] Haoyu Li, Xiangru Zhong, Bin Hu, and Huan Zhang. Two-stage learning of stabilizing neural controllers via zubov sampling and iterative domain expansion, 2025. URL <https://arxiv.org/abs/2506.01356>.
 - [53] Michael Y. Li, Erin Grant, and Thomas L. Griffiths. Gaussian process surrogate models for neural networks, 2023. URL <https://arxiv.org/abs/2208.06028>.
 - [54] Yuxi Li. Deep reinforcement learning: An overview, 2018. URL <https://arxiv.org/abs/1701.07274>.

- [55] Anirudha Majumdar and Russ Tedrake. Funnel libraries for real-time robust feedback motion planning, 2017. URL <https://arxiv.org/abs/1601.04037>.
- [56] Udayan Mandal, Guy Amir, Haoze Wu, Ieva Daukantas, Fletcher Lee Newell, Umberto J. Ravaioli, Baoluo Meng, Michael Durling, Milan Ganai, Tobey Shim, Guy Katz, and Clark Barrett. Formally verifying deep reinforcement learning controllers with lyapunov barrier certificates, 2024. URL <https://arxiv.org/abs/2405.14058>.
- [57] Daniele Masti, Filippo Fabiani, Giorgio Gnecco, and Alberto Bemporad. Counter-example guided inductive synthesis of control lyapunov functions for uncertain systems. *IEEE Control Systems Letters*, 7:2047–2052, 2023. ISSN 2475-1456. doi: 10.1109/lcsys.2023.3285102. URL <http://dx.doi.org/10.1109/LCSYS.2023.3285102>.
- [58] Jean Mawhin. *Alexandr Mikhailovich Liapunov, The general problem of the stability of motion (1892)*, pages 664–676. 01 2005.
- [59] Daniel Mellinger and Vijay Kumar. Minimum snap trajectory generation and control for quadrotors. In *2011 IEEE International Conference on Robotics and Automation*, pages 2520–2525, 2011. doi: 10.1109/ICRA.2011.5980409.
- [60] Yiming Meng and Jun Liu. Towards learning and verifying maximal lyapunov-barrier functions with a zubov pde formulation, 2025. URL <https://arxiv.org/abs/2511.09523>.
- [61] Andrea Peruffo, Daniele Ahmed, and Alessandro Abate. Automated and formal synthesis of neural barrier certificates for dynamical models, 2020. URL <https://arxiv.org/abs/2007.03251>.
- [62] Ping Qiao, Yizhong Wu, and Jianwan Ding. Optimal control of a black-box system based on surrogate models by spatial adaptive partitioning method. *ISA Transactions*, 100:63–73, 2020. ISSN 0019-0578. doi: <https://doi.org/10.1016/j.isatra.2019.11.012>. URL <https://www.sciencedirect.com/science/article/pii/S0019057819304926>.
- [63] Zengyi Qin, Kaiqing Zhang, Yuxiao Chen, Jingkai Chen, and Chuchu Fan. Learning safe multi-agent

- control with decentralized neural barrier certificates, 2021. URL <https://arxiv.org/abs/2101.05436>.
- [64] G.V. Raffo, Manuel Ortega, and Francisco Rubio. Backstepping/nonlinear h_∞ control for path tracking of a quadrotor unmanned aerial vehicle. pages 3356 – 3361, 07 2008. doi: 10.1109/ACC.2008.4587010.
- [65] Umberto J. Ravaioli, James Cunningham, John McCarroll, Vardaan Gangal, Kyle Dunlap, and Keri-anne L. Hobbs. Safe reinforcement learning benchmark environments for aerospace control systems. In *2022 IEEE Aerospace Conference (AERO)*, pages 1–20, 2022. doi: 10.1109/AERO53065.2022.9843750.
- [66] Hadi Ravanbakhsh and Sriram Sankaranarayanan. Counterexample-guided stabilization of switched systems using control lyapunov functions. In *Proceedings of the 18th International Conference on Hybrid Systems: Computation and Control*, HSCC ’15, page 297–298, New York, NY, USA, 2015. Association for Computing Machinery. ISBN 9781450334334. doi: 10.1145/2728606.2728647. URL <https://doi.org/10.1145/2728606.2728647>.
- [67] Chelsea Sidrane, Amir Maleki, Ahmed Irfan, and Mykel J. Kochenderfer. Overt: An algorithm for safety verification of neural network control policies for nonlinear systems, 2021. URL <https://arxiv.org/abs/2108.01220>.
- [68] Oswin So and Chuchu Fan. Solving stabilize-avoid optimal control via epigraph form and deep reinforcement learning, 2023. URL <https://arxiv.org/abs/2305.14154>.
- [69] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit Seshia, and Vijay Saraswat. Combinatorial sketching for finite programs. *SIGARCH Comput. Archit. News*, 34(5):404–415, October 2006. ISSN 0163-5964. doi: 10.1145/1168919.1168907. URL <https://doi.org/10.1145/1168919.1168907>.
- [70] Victor Talpaert, Ibrahim Sobh, B Ravi Kiran, Patrick Mannion, Senthil Yogamani, Ahmad El-Sallab, and Patrick Perez. Exploring applications of deep reinforcement learning for real-world autonomous driving systems, 2019. URL <https://arxiv.org/abs/1901.01536>.
- [71] Mukun Tong, Charles Dawson, and Chuchu Fan. Enforcing safety for vision-based controllers via

- control barrier functions and neural radiance fields, 2023. URL <https://arxiv.org/abs/2209.12266>.
- [72] Yusuke Tsuzuku, Issei Sato, and Masashi Sugiyama. Lipschitz-margin training: Scalable certification of perturbation invariance for deep neural networks, 2018. URL <https://arxiv.org/abs/1802.04034>.
- [73] Chengxiao Wang, Haoze Wu, and Gagandeep Singh. Formal synthesis of certifiably robust neural lyapunov-barrier certificates, 2026. URL <https://arxiv.org/abs/2602.05311>.
- [74] Junkai Wang, Yuxuan Zhao, Mi Zhou, and Fumin Zhang. Learning robust regions of attraction using rollout-enhanced physics-informed neural networks with policy iteration, 2025. URL <https://arxiv.org/abs/2508.19398>.
- [75] Shiqi Wang, Huan Zhang, Kaidi Xu, Xue Lin, Suman Jana, Cho-Jui Hsieh, and J. Zico Kolter. Beta-crown: Efficient bound propagation with per-neuron split constraints for complete and incomplete neural network robustness verification, 2021. URL <https://arxiv.org/abs/2103.06624>.
- [76] Yujia Wang and Zhe Wu. Control lyapunov-barrier function-based safe reinforcement learning for nonlinear optimal control. *AIChE Journal*, 70(3):e18306, 2024. doi: <https://doi.org/10.1002/aic.18306>. URL <https://aiche.onlinelibrary.wiley.com/doi/abs/10.1002/aic.18306>.
- [77] Tsui-Wei Weng, Huan Zhang, Hongge Chen, Zhao Song, Cho-Jui Hsieh, Duane Boning, Inderjit S. Dhillon, and Luca Daniel. Towards fast computation of certified robustness for relu networks, 2018. URL <https://arxiv.org/abs/1804.09699>.
- [78] Haoze Wu, Omri Isac, Aleksandar Zeljić, Teruhiro Tagomori, Matthew Daggitt, Wen Kokke, Idan Refaeli, Guy Amir, Kyle Julian, Shahaf Bassan, Pei Huang, Ori Lahav, Min Wu, Min Zhang, Ekaterina Komendantskaya, Guy Katz, and Clark Barrett. Marabou 2.0: A versatile formal analyzer of neural networks, 2024. URL <https://arxiv.org/abs/2401.14461>.
- [79] Junlin Wu, Andrew Clark, Yiannis Kantaros, and Yevgeniy Vorobeychik. Neural lyapunov control for discrete-time systems, 2023. URL <https://arxiv.org/abs/2305.06547>.

- [80] Xiangru Xu, Paulo Tabuada, Jessie W. Grizzle, and Aaron D. Ames. Robustness of control barrier functions for safety critical control**this work is partially supported by the national science foundation grants 1239055, 1239037 and 1239085. *IFAC-PapersOnLine*, 48(27):54–61, 2015. ISSN 2405-8963. doi: 10.1016/j.ifacol.2015.11.152. URL <http://dx.doi.org/10.1016/j.ifacol.2015.11.152>.
- [81] Lujie Yang, Hongkai Dai, Zhouxing Shi, Cho-Jui Hsieh, Russ Tedrake, and Huan Zhang. Lyapunov-stable neural control for state and output feedback: A novel formulation, 2024. URL <https://arxiv.org/abs/2404.07956>.
- [82] Yujie Yang, Yuxuan Jiang, Yichen Liu, Jianyu Chen, and Shengbo Eben Li. Model-free safe reinforcement learning through neural barrier certificate. *IEEE Robotics and Automation Letters*, 8(3): 1295–1302, 2023. doi: 10.1109/LRA.2023.3238656.
- [83] Dmitry Yarotsky. Error bounds for approximations with deep relu networks, 2017. URL <https://arxiv.org/abs/1610.01145>.
- [84] Hongzhan Yu, Chiaki Hirayama, Chenning Yu, Sylvia Herbert, and Sicun Gao. Sequential neural barriers for scalable dynamic obstacle avoidance, 2023. URL <https://arxiv.org/abs/2307.03015>.
- [85] Huan Zhang, Tsui-Wei Weng, Pin-Yu Chen, Cho-Jui Hsieh, and Luca Daniel. Efficient neural network robustness certification with general activation functions, 2018. URL <https://arxiv.org/abs/1811.00866>.
- [86] Songyuan Zhang and Chuchu Fan. Learning to stabilize high-dimensional unknown systems using lyapunov-guided exploration, 2024. URL <https://arxiv.org/abs/2306.08722>.
- [87] Songyuan Zhang, Yumeng Xiu, Guannan Qu, and Chuchu Fan. Compositional neural certificates for networked dynamical systems, 2023. URL <https://arxiv.org/abs/2303.14564>.
- [88] Yuyi Zhong, Ruiwei Wang, and Siau-Cheng Khoo. Expediting neural network verification via network reduction, 2023. URL <https://arxiv.org/abs/2308.03330>.
- [89] Ruikun Zhou, Thanin Quartz, Hans De Sterck, and Jun Liu. Neural lyapunov control of unknown nonlinear systems with stability guarantees, 2022. URL <https://arxiv.org/abs/2206.01913>.

- [90] Đorđe Žikelić, Mathias Lechner, Thomas A. Henzinger, and Krishnendu Chatterjee. Learning control policies for stochastic systems with reach-avoid guarantees, 2022. URL <https://arxiv.org/abs/2210.05308>.